Lecture Notes: Mathematical basics of ML

Scott Pesme
INRIA Grenoble

October 15, 2026

Contents

1 Introduction **3**

2 Prerequisites **4**
2.1 Definitions and notations 4
2.2 Linear Algebra 4
2.3 Span, rank, and generic full-rankness 5
2.4 Gradients 7
2.5 Convexity 7
2.6 Coercivity and existence of a minimum 8
2.7 Good practices and sanity checks 8

3 Learning framework **10**
3.1 What is "data"? 10
3.2 Supervised vs. unsupervised learning 10
3.3 Supervised learning: regression vs. classification 11

4 Regression **13**
4.1 The regression problem 13
4.2 Choosing the function class 14
4.3 Choosing the loss function 15
4.4 Minimising the empirical risk 16

5 Least squares **16**
5.1 Underparametrised setting ($n \geq d$): a unique solution 18
5.2 Overparametrised setting ($d > n$): an infinite space of solutions 18
5.3 Gradient descent and stochastic gradient descent 20
5.4 Ridge regression (weight decay) 24
5.5 Feature expansion 26

6 Classification **28**
6.1 Binary classification 28
6.2 Choosing a loss: from the ideal one to convex surrogates 29
6.3 Logistic regression 30
6.4 Separable and non-separable data 32
6.5 Multiclass classification 35

7 Unsupervised learning **38**
7.1 The setup, and what changes without labels 38
7.2 Principal component analysis 39
7.3 K-means 43

8 Exercises **47**

A Useful Formulas and Good Practices **49**
A.1 Centring and standardising the features 49
A.2 Numerical stability 50

1 Introduction

Slides of the general introduction can be found here.

2 Prerequisites

This section gathers the notations, definitions and mathematical facts that are used throughout these notes. Feel free to skip it on a first read and to come back to it whenever a statement is invoked in the text.

2.1 Definitions and notations

Keep in mind that these are the notations I like to use, but these are obviously personal and others will use different ones!!

- the notation x will always be used for input data. E.g. $x_1, \dots, x_n \in \mathbb{R}^d$ could be some dataset. n is then the number of training samples, and d the dimension of each data point. Similarly, y will be used for output data, e.g. $y_1, \dots, y_n \in \mathbb{R}$ (or $\in \{0, 1\}$) are output data (or labels). When $d > n$, we often say that we are in the *over-parametrised setting* (also called *under-determined setting*, which can be confusing!), and when $d \leq n$ we say that we are in the *under-parametrised setting* (also called *over-determined setting*).
- $X = \begin{pmatrix} - & x_1 & - \\ & \vdots & \\ - & x_n & - \end{pmatrix} \in \mathbb{R}^{n \times d}$ corresponds to data / feature / design / observation matrix. It has n rows (the *number of samples*) and d columns (the *dimension of the data points*).
- $y = (y_1, \dots, y_n) \in \mathbb{R}^n$ corresponds to the output vector.
- for a vector $u \in \mathbb{R}^d$, we let $\|u\|_2 \coloneqq \sqrt{\sum_{j=1}^d u_j^2}$ denote the Euclidean norm of u (also called ℓ_2-norm). It is the only norm we use, so we drop the subscript and simply write $\|u\|$ from here on.
- for vectors $u, v \in \mathbb{R}^d$, we denote $\langle u, v \rangle \coloneqq \sum_{j=1}^d u_j v_j$ to be the inner product of u and v. Notice that $\|u\|^2 = \langle u, u \rangle$. Two vectors are said to be orthogonal if their inner product is null.
- the span of a family of vectors and the rank of a matrix are defined in Section 2.3.
- the null space (sometimes called kernel) of a matrix $A \in \mathbb{R}^{n \times d}$ is defined as $\mathrm{Ker}(A) = \{w \in \mathbb{R}^d, Aw = 0\}$.
- a matrix $A \in \mathbb{R}^{d \times d}$ is said to be symmetric if $A^\top = A$. It is said to be positive semi-definite (we write this as $A \succeq 0$) if for all vector $u \in \mathbb{R}^d$, $u^\top A u \geq 0$. For a symmetric matrix, this is equivalent to saying that all the eigenvalues of A are non-negative.

2.2 Linear Algebra

- For a matrix $A \in \mathbb{R}^{d \times d}$, it holds that $w^\top A w = \sum_{j,j'=1}^d w_j w_{j'} A_{j,j'}$.
- It holds that $X^\top X = \sum_{i=1}^n x_i x_i^\top \in \mathbb{R}^{d \times d}$ and $X X^\top = \left(\langle x_i, x_{i'} \rangle\right)_{1 \leq i, i' \leq n} \in \mathbb{R}^{n \times n}$.

- For a vector $w \in \mathbb{R}^d$, the vector Xw lives in $\mathbb{R}^n$ and is equal to $Xw = \begin{pmatrix} \langle x_1, w \rangle \\ \vdots \\ \langle x_n, w \rangle \end{pmatrix}$
- Let $L(w) = \frac{1}{2n} \sum_{i=1}^n (y_i - \langle w, x_i \rangle)^2$, then $L(w) = \frac{1}{2n} \|y - Xw\|^2$ and $\nabla L(w) = \frac{1}{n} X^\top (Xw - y)$.
- invertibility of $X^\top X$ and of $XX^\top$ is settled by the rank of X; see Section 2.3 for the precise statements and for why, on "typical" data, one of the two is always invertible.
- *Rank-nullity theorem:* for $A \in \mathbb{R}^{n \times d}$, it holds that $\operatorname{rank}(A) + \dim \operatorname{Ker}(A) = d$.
- *Trace.* For a square matrix $A \in \mathbb{R}^{d \times d}$, $\operatorname{tr}(A) = \sum_{j=1}^d A_{j,j}$ is the sum of its diagonal entries, equivalently the sum of its eigenvalues. It is linear, satisfies $\operatorname{tr}(A^\top) = \operatorname{tr}(A)$, and is *cyclic*: $\operatorname{tr}(AB) = \operatorname{tr}(BA)$ whenever both products make sense, even when A and B are rectangular of different shapes. Cyclicity is the workhorse of Section 7.
- *Frobenius norm.* For $A \in \mathbb{R}^{n \times d}$, $\|A\|_F^2 := \sum_{i,j} A_{i,j}^2 = \operatorname{tr}(A^\top A)$: the Euclidean norm of A read as a vector of nd entries. If the rows of A are $a_1, \ldots, a_n$ then $\|A\|_F^2 = \sum_{i=1}^n \|a_i\|^2$.
- *Orthogonal projectors.* A matrix $P \in \mathbb{R}^{d \times d}$ with $P^\top = P$ and $P^2 = P$ is the orthogonal projector onto its range. It satisfies $\|Pu\| \leq \|u\|$ for every u, and $\operatorname{tr}(P) = \operatorname{rank}(P)$, i.e. the trace of a projector is the dimension of the subspace it projects onto.

2.3 Span, rank, and generic full-rankness

This subsection gathers a few linear-algebra facts that come back throughout the course, together with a genericity statement explaining why, for "typical" data, either $X^\top X$ or $XX^\top$ is invertible.

Definition (Span and linear independence)**.** The span of vectors $u_1, \ldots, u_k \in \mathbb{R}^m$ is the linear subspace they generate,

$$\operatorname{span}(u_1, \ldots, u_k) := \left\{ \sum_{i=1}^k \lambda_i u_i \ : \ \lambda_1, \ldots, \lambda_k \in \mathbb{R} \right\} \subseteq \mathbb{R}^m.$$

The vectors are said to be linearly independent if none of them lies in the span of the others, equivalently if $\sum_i \lambda_i u_i = 0$ if and only if $\lambda_1 = \cdots = \lambda_k = 0$.

Definition (Rank)**.** The rank of a matrix $X \in \mathbb{R}^{n \times d}$ is the dimension of the span of its columns, which always coincides with the dimension of the span of its rows. In particular

$$\operatorname{rank}(X) \leq \min(n, d),$$

and X is said to be *full rank* when this bound is attained. We speak of *full column rank* when $\operatorname{rank}(X) = d$ (the columns are independent, which forces $n \geq d$) and of *full row rank* when $\operatorname{rank}(X) = n$ (the rows are independent, which forces $d \geq n$).

Two identities make the rank the right tool for deciding invertibility of the Gram matrices $X^\top X$ and $XX^\top$.

Lemma 2.1 (Kernel and rank of the Gram matrices)**.** *For any $X \in \mathbb{R}^{n\times d}$,*

$$\mathrm{Ker}(X^\top X) = \mathrm{Ker}(X), \qquad \mathrm{rank}(X^\top X) = \mathrm{rank}(XX^\top) = \mathrm{rank}(X).$$

Consequently:

- *$X^\top X \in \mathbb{R}^{d\times d}$ is invertible $\iff$ $\mathrm{rank}(X) = d$ (full column rank), which requires $n \geq d$;*
- *$XX^\top \in \mathbb{R}^{n\times n}$ is invertible $\iff$ $\mathrm{rank}(X) = n$ (full row rank), which requires $d \geq n$.*

Proof sketch. If $X^\top Xw = 0$, then $\|Xw\|^2 = w^\top X^\top Xw = 0$, so $Xw = 0$; the reverse inclusion is immediate, hence $\mathrm{Ker}(X^\top X) = \mathrm{Ker}(X)$. The rank equalities then follow from the rank-nullity theorem (Section 2.2). Finally, a square matrix is invertible if and only if its kernel is trivial, i.e. if and only if it is full rank. □

Lemma 2.2 (Generic full-rankness)**.** *Suppose the rows $x_1, \ldots, x_n \in \mathbb{R}^d$ of X are drawn* independently *from a distribution that is absolutely continuous with respect to the Lebesgue measure on $\mathbb{R}^d$ (i.e. it admits a density). Then, almost surely,*

$$\mathrm{rank}(X) = \min(n, d),$$

that is, X is full rank.

Why this holds. Full-rankness fails exactly when every maximal square submatrix of X has zero determinant. Each such determinant is a nonzero polynomial in the entries of X, and the zero set of a nonzero polynomial has Lebesgue measure zero in $\mathbb{R}^{n\times d}$. Since the rows are independent and each has a density, the matrix X itself has a density on $\mathbb{R}^{n\times d}$ (the product of the densities of its rows), and therefore assigns probability zero to any measure-zero set: the "degenerate" configurations occur with probability zero. Independence matters: if for instance all the rows were equal to the same random vector, each row would still have a density, but X would have rank 1. □

Combining the two lemmas and separating according to the shape of X gives the statement we use in practice.

Corollary 2.3 (Which Gram matrix is invertible?)**.** *Under the assumptions of the previous lemma, almost surely:*

- *if $n \leq d$ (in particular the* over-parametrised *case $d > n$), the rows are linearly independent,* $\mathrm{rank}(X) = n$*, and $XX^\top \in \mathbb{R}^{n\times n}$ is invertible;*
- *if $n \geq d$ (the* under-parametrised *case), the columns are linearly independent,* $\mathrm{rank}(X) = d$*, and $X^\top X \in \mathbb{R}^{d\times d}$ is invertible.*

When $n = d$ both hold simultaneously.

One direction, however, needs no genericity assumption at all, and is worth keeping in mind whenever $d > n$.

Lemma 2.4 (If $d > n$, then $X^\top X$ is never invertible)**.** *If $d > n$, then for* every $X \in \mathbb{R}^{n\times d}$ *we have*

$$\operatorname{rank}(X^\top X) = \operatorname{rank}(X) \leq n < d,$$

so $X^\top X \in \mathbb{R}^{d\times d}$ is not full rank, and hence not invertible*. Its null space is therefore nontrivial:*

$$\dim \operatorname{Ker}(X^\top X) = \dim \operatorname{Ker}(X) = d - \operatorname{rank}(X) \geq d - n > 0.$$

In words: in the over-parametrised setting there is always a nonzero direction w with $Xw = 0$, so the normal equations $X^\top Xw = X^\top y$ cannot have a unique solution.

2.4 Gradients

If $f : \mathbb{R}^d \to \mathbb{R}$ is twice differentiable, then its gradient $\nabla f : \mathbb{R}^d \to \mathbb{R}^d$ and Hessian $\nabla^2 f : \mathbb{R}^d \to \mathbb{R}^{d\times d}$ are defined as:

$$\nabla f(w) = \Big(\frac{\partial f}{\partial w_j}(w)\Big)_{1\leq j\leq d} \quad \nabla^2 f(w) = \Big(\frac{\partial^2 f}{\partial w_j \partial w_{j'}}(w)\Big)_{1\leq j,j'\leq d}$$

- For a vector $b \in \mathbb{R}^d$, the gradient of the linear function $f : \mathbb{R}^d \to \mathbb{R}$, $f(w) = \langle b, w\rangle$ is equal to $\nabla f(w) = b$.
- For a (not necessarily symmetric) matrix $A \in \mathbb{R}^{d\times d}$, $f(w) = \frac{1}{2} w^\top Aw$ is a quadratic function. Its gradient is $\nabla f(w) = \frac{1}{2}(A + A^\top)w$, which is equal to Aw if and only if A is a symmetric matrix. The Hessian of f is equal to $\nabla^2 f(w) = \frac{1}{2}(A + A^\top)$.

2.5 Convexity

A function $f : \mathbb{R}^d \to \mathbb{R}$ is said to be convex if for all $w_1, w_2 \in \mathbb{R}^d$ and $\lambda \in [0, 1]$, $f(\lambda w_1 + (1 - \lambda)w_2) \leq \lambda f(w_1) + (1 - \lambda)f(w_2)$ (DO A DRAWING TO VISUALISE THIS!). It is said to be *strictly convex* if this inequality is strict whenever the two points differ, i.e. for all $w_1 \neq w_2$ and all $\lambda \in (0, 1)$, $f(\lambda w_1 + (1 - \lambda)w_2) < \lambda f(w_1) + (1 - \lambda)f(w_2)$.

- if f is convex and differentiable, then for all $w_1, w_2 \in \mathbb{R}^d$, it holds that $f(w_2) \geq f(w_1) + \langle \nabla f(w_1), w_2 - w_1\rangle$ (DO A DRAWING TO VISUALISE THIS!).
- if f is convex, then all local minima are global. Moreover, if f is differentiable and $\nabla f(w^\star) = 0$, then $w^\star$ is a global minimum: the inequality of the previous bullet immediately gives $f(w) \geq f(w^\star) + \langle \nabla f(w^\star), w - w^\star\rangle = f(w^\star)$ for every w. However, keep in mind that there exist convex functions which do not have any minima! (the exponential function for example)
- if f is strictly convex, then it has *at most one* minimiser: a minimum need not exist (again think of the exponential), but if it does, it is unique. Indeed, if $w_1 \neq w_2$ were two minimisers with common value m, then strict convexity at $\lambda = \frac{1}{2}$ would give $f(\frac{w_1+w_2}{2}) < \frac{1}{2} f(w_1) + \frac{1}{2} f(w_2) = m$, contradicting the minimality of m.
- if a function f is convex, then its sublevel sets $\{w \in \mathbb{R}^d, f(w) \leq c\}$ are convex sets for all $c \in \mathbb{R}$. The converse is false (for example think of $x \mapsto \sqrt{|x|}$ in 1d).
- let $\lambda_1, \lambda_2 \geq 0$, if f_1 and f_2 are convex functions, then so is $\lambda_1 f_1 + \lambda_2 f_2$.

- if $f : \mathbb{R}^n \to \mathbb{R}$ is convex and $A \in \mathbb{R}^{n\times d}$ and $b \in \mathbb{R}^n$, then $w \in \mathbb{R}^d \mapsto f(Aw + b)$ is convex too.

2.6 Coercivity and existence of a minimum

Definition (Coercive function)**.** A function $f : \mathbb{R}^d \to \mathbb{R}$ is said to be *coercive* if

$$f(w) \xrightarrow[\|w\|\to+\infty]{} +\infty,$$

i.e. for every $M \in \mathbb{R}$ there exists a radius $R > 0$ such that $f(w) \geq M$ whenever $\|w\| \geq R$. In words: f blows up in every direction as we move away from the origin.

For example, $w \mapsto \|w\|^2$ is coercive, whereas $w \mapsto e^w$ (on $\mathbb{R}$) and $(w_1, w_2) \mapsto w_1^2$ (on $\mathbb{R}^2$) are not: the first one stays bounded as $w \to -\infty$, the second one is constant along the w_2-axis.

Proposition 2.5 (Continuous coercive functions admit a minimum)**.** *If $f : \mathbb{R}^d \to \mathbb{R}$ is continuous and coercive, then f admits a (global) minimum: there exists $w^\star \in \mathbb{R}^d$ such that $f(w^\star) \leq f(w)$ for all $w \in \mathbb{R}^d$.*

Proof. Fix any point $w_0 \in \mathbb{R}^d$ and consider the sublevel set $K = \{w \in \mathbb{R}^d : f(w) \leq f(w_0)\}$. It is non-empty (it contains w_0), closed (as the preimage of the closed set $(-\infty, f(w_0)]$ by the continuous function f), and bounded (by coercivity, there is an R such that $f(w) > f(w_0)$ as soon as $\|w\| \geq R$, so K is contained in the ball of radius R). Hence K is compact, and the continuous function f attains its minimum on K at some $w^\star \in K$. Finally, any $w \notin K$ satisfies $f(w) > f(w_0) \geq f(w^\star)$, so $w^\star$ is in fact a global minimiser of f on the whole of $\mathbb{R}^d$. □

Note that coercivity only gives *existence*: the minimum need not be unique (think of $w \mapsto (\|w\|^2 - 1)^2$, minimised on the whole unit sphere). Combined with strict convexity (see Section 2.5), however, it gives existence *and* uniqueness.

2.7 Good practices and sanity checks

Math sanity checks Always check that the math makes sense!

- If $f : \mathbb{R}^d \to \mathbb{R}$, then for a vector $w \in \mathbb{R}^d$, $\nabla f(w)$ must belong to $\mathbb{R}^d$! So for example if $f(w) = \frac{1}{2}\|w\|^2$, then writing that "$\nabla f(w) = \|w\|$" doesn't make sense.
- Check that the matrix operations are allowed: if $L(w) = \|y - Xw\|^2$, writing that "$\nabla L(w) = X(Xw - y)$" doesn't make sense because the operations XX and Xy don't make sense for $n \neq d$ (and also because of the remark right above).

Dimensional sanity check. Even if an expression is mathematically well-formed, it might be meaningless from the point of view of "units" or "dimensions." A quick check is to make sure your formulas are *homogeneous*: every term you add or compare should have the same "type."

Example. Suppose our data are temperature observations y_i measured in degrees Celsius. A feature vector $x \in \mathbb{R}^d$ could represent input quantities such as:

$$x = (\text{[altitude in meters]}, \text{[pressure in pascals]}, \text{[wind speed in meters per second]}).$$

A parameter vector $w \in \mathbb{R}^d$ scales each feature so that the inner product $\langle w, x \rangle$ has the same unit as y (degrees Celsius). In our small example the units of w are

$$([^\circ\text{C}] \cdot [\text{meters}]^{-1}, [^\circ\text{C}] \cdot [\text{pascals}]^{-1}, [^\circ\text{C}] \cdot [\text{meters}]^{-1} \cdot [\text{seconds}])$$

Now notice that:

- $\langle w, x \rangle$ makes sense: each coordinate of w carries the reciprocal unit of the corresponding coordinate of x, so the sum yields something in degrees Celsius.
- $w + x$ does *not* make sense: w and x do not have the same units (adding "degrees per meter" to "meters" is meaningless).

3 Learning framework

Before diving into any algorithm, let us informally describe what we mean by *learning* from data. The guiding idea of machine learning is to replace explicit, hand-written rules by a procedure that *extracts patterns from examples*. Rather than telling a computer exactly how to recognise a cat, or how to price a flat, we provide it with a collection of *data* and let it *learn* a rule that performs well and, crucially, keeps performing well on new data it has never seen. This section sets up the vocabulary; the precise definitions and the mathematics come in the following sections.

3.1 What is "data"?

Everything starts from *data*: a collection of examples $x_1, \dots, x_n$, where n is the number of samples. Throughout these notes we represent each example as a vector (sometimes called "sample" or "input") x_i, but this vector can encode essentially anything, provided we agree on a way to turn the raw object into numbers. A few concrete cases:

- *Feature vectors.* The most direct case, where each coordinate is an interpretable measurement. To describe a flat, one might take

$$x = (\text{[surface in m}^2\text{]},\ \text{[number of rooms]},\ \text{[floor]},\ \dots).$$

- *Images.* A colour image of $H \times W$ pixels is a grid of RGB intensities. Flattening it gives a vector $x \in \mathbb{R}^d$ with $d = 3HW$ (one coordinate per colour channel and pixel). A modest 224×224 image already lives in dimension $d \approx 1.5 \times 10^5$.

- *Sound.* An audio clip is an amplitude sampled through time. If we record d amplitudes, the clip is a vector $x \in \mathbb{R}^d$, one coordinate per time step.

- *Text, graphs, ...* Similarly, a piece of text, a molecule, or a social network can all be encoded as vectors once we fix a suitable representation.

The common thread is that, after a suitable encoding, every input becomes a point $x_i \in \mathbb{R}^d$. This is what lets a single mathematical framework cover such different-looking objects.

In *supervised* problems each input x_i comes with a *label* y_i, and the nature of y varies just as much. It may be a real number $y \in \mathbb{R}$ (a price, a temperature), a binary label $y \in \{0, 1\}$ (spam or not), one of several categories $y \in \{1, \dots, K\}$ (which digit is written), or even a structured object such as another image or a sentence. The two basic regimes, real-valued and categorical labels, are exactly the regression and classification problems we distinguish below.

3.2 Supervised vs. unsupervised learning

The first thing to ask about a learning problem is *what the data looks like*. This largely determines which family of methods is relevant.

Supervised learning. Here each data point comes as a *pair* (x, y): an *input* (or feature vector) x together with a *label* (or response) y that we would like to predict. The training set is a list

$$(x_1, y_1), \dots, (x_n, y_n),$$

and the goal is to learn a function f such that $f(x) \approx y$, so that we can predict the label of a *new* input. The name "supervised" reflects the fact that, for every training input,

a supervisor has provided the correct answer y. Predicting the price of a flat from its characteristics, or deciding whether an email is spam, are typical examples.

Unsupervised learning. Now the data consists of *inputs only*,

$$x_1, \dots, x_n,$$

with no labels attached. Since there is nothing specific to predict, the goal shifts to *discovering structure* in the data on its own. Typical tasks include *clustering* (grouping similar points together) and *dimensionality reduction* (finding a few directions that capture most of the variability). We will see these in Section 7, through the K-means algorithm and principal component analysis respectively.

Self-supervised learning. A particularly powerful idea, and the engine behind most modern large-scale models, is to *manufacture the labels from the unlabelled data itself.* In *self-supervised* learning one starts from inputs only and creates an artificial supervised problem by hiding part of each data point and asking the model to predict it from the rest. No human annotation is required, yet the resulting task looks supervised. The typical example is language modelling: take a sentence, hide a word (or simply ask for the next one), and predict it from its surrounding context; repeating this over enormous text corpora is how large language models are trained. The same recipe applies to images (predict missing patches) or audio. Having learned rich representations this way, the model can then be adapted to an actual downstream task using comparatively few labels.

> **Remark** (Other paradigms)**.** *Supervised and unsupervised learning are the two pillars of this course, but they do not exhaust machine learning. In* reinforcement learning*, for instance, an agent interacts with an environment and learns from* rewards *rather than from labelled examples: think of a program learning to play a game by trial and error. We will not treat it here.*

3.3 Supervised learning: regression vs. classification

Within supervised learning, problems are further split according to the *nature of the label* y we want to predict.

Regression. When the label is a *real-valued* (continuous) quantity, $y \in \mathbb{R}$, we speak of a *regression* problem. Predicting a price, a temperature, or a person's age from a photograph are regression tasks. This is the setting we develop in detail in the next section.

Classification. When the label takes its values in a *finite set of categories*, we speak of a *classification* problem. With two categories, $y \in \{0, 1\}$ (or $\{-1, +1\}$), the problem is called *binary* classification, e.g. spam vs. not spam, healthy vs. sick. With more than two categories, $y \in \{1, \dots, K\}$, it is *multi-class*, e.g. recognising which digit, from 0 to 9, is written in an image.

> **Remark** (Same recipe, different label)**.** *Regression and classification share the very same blueprint: a class of candidate functions, a way to measure the quality of a prediction, and an algorithm that searches for a good function. What changes is mostly the type of y and, consequently, the loss used to compare a prediction with the truth.*

> *Mastering regression therefore gives us most of the tools needed for classification as well.*

4 Regression

4.1 The regression problem

In a *regression* problem we want to predict a real-valued quantity from some input. The input is described by a *feature vector* $x \in \mathcal{X} \subseteq \mathbb{R}^d$, and the quantity we want to predict is a *response* $y \in \mathbb{R}$.

Example 4.1. *Suppose we want to predict the price of a flat. Each flat can be encoded by a feature vector*

$$x = (\text{[surface in } m^2\text{]},\ \text{[number of rooms]},\ \text{[floor]},\ \dots) \in \mathbb{R}^d,$$

and the response $y \in \mathbb{R}$ is its price. Each coordinate of x is a feature *(also called* covariate *or* explanatory variable*), and d is the number of features.*

We are given a *training set* of n pairs

$$(x_1, y_1), \dots, (x_n, y_n) \in \mathbb{R}^d \times \mathbb{R},$$

where n is the number of training samples and d the dimension of each data point. As in the rest of these notes, we stack the inputs into the *design matrix* and the outputs into a single vector:

$$X = \begin{pmatrix} - & x_1 & - \\ & \vdots & \\ - & x_n & - \end{pmatrix} \in \mathbb{R}^{n \times d}, \qquad y = (y_1, \dots, y_n) \in \mathbb{R}^n.$$

Remark (Encoding non-numerical features)**.** *We have implicitly assumed that each input is a vector of real numbers, $x \in \mathbb{R}^d$. In practice many features are* categorical *rather than continuous: a person's sex (*`male`*/*`female`*), a colour, a city, etc. We turn them into numbers as follows.*

- *A* binary *feature can be encoded by a single $-1/1$ indicator, e.g.* `male` $\mapsto -1$ *and* `female` $\mapsto 1$*.*
- *A categorical feature with $K \geq 3$ possible values should* not *simply be mapped to $1, 2, \dots, K$: this would invent a spurious order and notion of distance between categories (it would suggest that "city 3" lies between "city 2" and "city 4"). Instead one uses* one-hot *(or* dummy*) encoding: the feature is replaced by K binary features, exactly one of which equals 1. For example the colour* {`red`, `green`, `blue`} *becomes*

  $$\texttt{red} \mapsto (1,0,0), \quad \texttt{green} \mapsto (0,1,0), \quad \texttt{blue} \mapsto (0,0,1).$$

  One might object that for $K = 2$ the one-hot encoding does not give back the "$-1/1$" encoding of the previous item. It is in fact (in a certain sense) equivalent to it.

After such an encoding every input is again a genuine vector $x \in \mathbb{R}^d$, and the whole framework below applies unchanged.

The goal is to *learn* a function $f : \mathcal{X} \to \mathbb{R}$ such that

$$f(x) \approx y \qquad \text{for unseen pairs } (x, y).$$

Once such an f is found, we can predict the response of a brand new input x simply by evaluating $f(x)$.

What do we need to specify? Turning the vague wish "$f(x) \approx y$" into a well-posed problem requires two ingredients:

1. a *class of functions* $\mathcal{F}$ in which we look for f (e.g. all linear functions of x);
2. a way to *quantify* how good a candidate f is on the data, i.e. a *loss function*.

4.2 Choosing the function class

The function class $\mathcal{F}$ collects all the candidate predictors we are willing to consider. In practice we never search over an abstract set of functions: instead we fix a *parametrisation*, that is, we describe every candidate by a finite vector of *parameters* (or *weights*) $w \in \mathbb{R}^p$ (where p corresponds to the number of trainable parameters) and denote the corresponding predictor by f_w. The function class is then

$$\mathcal{F} = \{\, f_w \;:\; w \in \mathbb{R}^p \,\},$$

and *learning the function reduces to searching for a good parameter* w. How w enters f_w is a modelling choice; here are the main examples. (For the linear models that occupy the rest of these notes the parameter has exactly one coordinate per feature, so it lives in the same $\mathbb{R}^d$ as the inputs; the last two examples below are parametrised differently, and there w simply collects however many weights the model has.)

Linear in the input. The simplest choice makes the prediction a linear function of the input,

$$f_w(x) = \langle w, x \rangle = \sum_{j=1}^{d} w_j x_j, \qquad w \in \mathbb{R}^d,$$

so that the parameter w has the same dimension d as the input x (i.e., $p = d$).

Nonlinear in the parameters. Nothing forces w to enter linearly. One may equally consider parametrisations in which the parameter appears *inside* a nonlinear function: the resulting objective is typically non-convex in w, and must be tackled by iterative optimisation (e.g. gradient descent), with no guarantee of reaching a global minimum.

Neural networks. The most prominent and expressive parametrisation that is nonlinear in w is a *neural network*. In its simplest, one-hidden-layer form,

$$f_w(x) = \sum_{k=1}^{m} a_k \, \sigma\big(\langle b_k, x \rangle + c_k\big),$$

where σ is a fixed nonlinearity (e.g. a ReLU or a sigmoid) and the parameters $w = (a_k, b_k, c_k)_{k=1}^{m}$ gather all the weights. Increasing the width m (or stacking several such layers) yields an extremely rich function class. The price to pay is, once more, a highly non-convex dependence on w, which is the central difficulty of modern deep learning.

Exercise 4.2 ($\star$)**.** For the one-hidden-layer neural network above, with inputs $x \in \mathbb{R}^d$ and width m, what is the number p of trainable parameters?

Why a continuous parameter space matters. Whatever the parametrisation, the key point is that the search space is $\mathbb{R}^p$, a *continuous* space. This is what makes learning amenable to optimisation: we can talk about how $L(w)$ changes when w is perturbed by a small amount, compute gradients $\nabla L(w)$, and move w step by step in a direction that decreases the loss. If instead the candidates were indexed by a *discrete* set (say, $w \in \{0,1\}^p$, or a finite list of hand-crafted rules), there would be no notion of gradient nor of "small step", and we would be left with combinatorial search over exponentially many candidates, which is in general much harder.

4.3 Choosing the loss function

Fixing a function class is not enough: we still need to say what it means for a prediction to be *good*. This is the role of a *loss function* $\ell(\hat{y}, y)$, which quantifies how much we penalise predicting $\hat{y} = f(x)$ when the true response is y. Averaging the loss over the training set gives the *empirical risk*, which we view as a function of the parameter w and denote

$$L(w) = \frac{1}{n} \sum_{i=1}^{n} \ell(f_w(x_i), y_i).$$

Empirical risk minimisation then consists in choosing the parameter w (equivalently, $f_w \in \mathcal{F}$) so as to make $L(w)$ as small as possible.

Remark (The constant $1/n$ does not matter for the minimiser)**.** *Multiplying an objective by a positive constant does not move its minimisers: for any $c > 0$,* $\operatorname{argmin}_w c\,L(w) = \operatorname{argmin}_w L(w)$. *In particular the factor $1/n$ in front of the sum is immaterial when our only goal is to* locate *the best w. We nonetheless keep it throughout these notes, because it makes $L(w)$ an* average *loss per sample rather than a total: its scale then does not grow with the size of the training set, and losses computed on datasets of different sizes remain comparable. Be aware, though, that the constant* does *rescale the gradient ∇L, and hence matters once a step size enters the picture, as in gradient descent below. (The extra $\frac{1}{2}$ that will appear for the squared error is a separate matter: it belongs to the loss ℓ itself and is there only to cancel the 2 coming from the derivative.)*

Several ways to penalise an error. The choice of ℓ is itself a modelling decision: different losses encode different ideas of what a good fit is. A few common choices are

- the *squared error* $\ell(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2$, by far the most common, which strongly penalises large residuals;
- the *absolute error* $\ell(\hat{y}, y) = |\hat{y} - y|$, more robust to outliers but non-differentiable at 0;
- the *Huber loss*, a compromise that is quadratic for small residuals and linear for large ones.

In the remainder of this section we focus on the squared error, which, when combined with a model that is linear in w, yields the particularly clean *least-squares* problem.

Remark (Training error is not the goal)**.** *Making the empirical risk small only guarantees that f fits the* training *data. What we actually care about is the* test error*, i.e. how well f predicts on new, unseen inputs. A function can fit the training set perfectly and still generalise poorly.*

4.4 Minimising the empirical risk

Choosing a function class and a loss has turned learning into an optimisation problem:

$$w^\star \in \operatorname*{argmin}_{w \in \mathbb{R}^p} \; L(w),$$

where $\operatorname{argmin}_w L(w)$ denotes the set of parameters at which L attains its minimum, which is not necessarily unique (and may not even exist as we will see later on in the classification setting).

For most models and losses there is no formula for the minimiser, so we approximate $w^\star$ by an *iterative* method that only needs to evaluate gradients of L. The basic idea is *gradient descent*: from some starting point w_0, repeatedly take a small step opposite to the gradient,

$$w_{t+1} = w_t - \gamma \, \nabla L(w_t),$$

and its cheaper *stochastic* variant, which at each step uses the gradient at only a few sampled data points. We will come back to both in detail in Section 5.3; for now it is enough to keep in mind that minimising L is generally done by such gradient-based iteration.

5 Least squares

We now specialise the two ingredients above to the simplest case:

$$f_w(x) = \langle w, x \rangle = \sum_{j=1}^{d} w_j x_j, \qquad w \in \mathbb{R}^d,$$

together with the squared-error loss. The function class is thus

$$\mathcal{F} = \{\, x \mapsto \langle w, x \rangle \;:\; w \in \mathbb{R}^d \,\}.$$

Note that in this setting the number of trainable parameters coincides with the dimension of the inputs: $p = d$. This is why, throughout this section, the parameter w and the inputs x_i live in the same space $\mathbb{R}^d$, and why comparing n with d amounts to comparing the number of samples with the number of parameters.

Remark (Adding an intercept)**.** *Often we want an* affine *predictor $f(x) = \langle w, x \rangle + b$ with an intercept (or bias) $b \in \mathbb{R}$, so that $f(0) \neq 0$ is allowed. This fits into the same framework: simply append a constant coordinate equal to 1 to each input, $\tilde{x} = (x, 1) \in \mathbb{R}^{d+1}$, and a coordinate b to the weights, $\tilde{w} = (w, b) \in \mathbb{R}^{d+1}$. Then $\langle \tilde{w}, \tilde{x} \rangle = \langle w, x \rangle + b$, and we are back to a purely linear model in dimension $d+1$. For this reason we keep the notation $f_w(x) = \langle w, x \rangle$ below without loss of generality.*

The least-squares objective. Plugging the linear model into the empirical risk with the square loss, learning amounts to minimising over $w \in \mathbb{R}^d$ the function

$$L(w) = \frac{1}{2n} \sum_{i=1}^{n} \left(\langle w, x_i \rangle - y_i \right)^2 = \frac{1}{2n} \|Xw - y\|^2,$$

where the second equality follows from stacking the residuals $\langle w, x_i \rangle - y_i$ into the vector $Xw - y \in \mathbb{R}^n$.

> **Remark.** *Notice that L is a non-negative function, and that $L(w) = 0$ if and only if $Xw = y$, i.e. if and only if the vector w perfectly fits the dataset: $\langle x_i, w \rangle = y_i$ for all $i \in \{1, \dots, n\}$.*

The problem

$$w^\star \in \operatorname*{argmin}_{w \in \mathbb{R}^d} \frac{1}{2n} \|Xw - y\|^2$$

is called the *least-squares* problem.

> **Exercise 5.1** ($\star\star$)**.** Check that we indeed have that
>
> $$\frac{1}{2n} \sum_{i=1}^{n} \left(\langle w, x_i \rangle - y_i \right)^2 = \frac{1}{2n} \|Xw - y\|^2.$$
>
> Then write L as an explicit quadratic in w:
>
> $$L(w) = \tfrac{1}{2n}\, w^\top X^\top X\, w - \tfrac{1}{n}\, w^\top X^\top y + \tfrac{1}{2n} \|y\|^2.$$
>
> Deduce that $\nabla L(w) = \frac{1}{n} X^\top (Xw - y)$ and that the Hessian is $\nabla^2 L(w) = \frac{1}{n} X^\top X$, independently of w.

Solving the least squares problem. The function L is convex (it is a quadratic with positive semi-definite Hessian $\frac{1}{n} X^\top X \succeq 0$, see Section 2.5), so any point where the gradient vanishes is a global minimum. Computing the gradient of L (see Section 2.2) leads to:

$$\nabla L(w) = \tfrac{1}{n}\, X^\top (Xw - y).$$

Setting $\nabla L(w^\star) = 0$, the factor $\frac{1}{n}$ cancels and we obtain the *normal equations*

$$X^\top X\, w^\star = X^\top y.$$

From here, we distinguish two different settings:

- The *under-parametrised* setting (sometimes called the *over-determined* setting), where there are fewer trainable parameters than data samples, i.e. $d \leq n$. In this setting, we will see that the normal equations admit a *unique* solution[1], and hence L has a unique global minimiser.
- The *over-parametrised* setting (sometimes called the *under-determined* setting), where there are more trainable parameters than data samples, i.e. $d > n$. In this setting, we will see that the normal equations admit *infinitely many* solutions, and hence L has infinitely many global minimisers.

[1] Under the mild assumption that X has full column rank.

5.1 Underparametrised setting ($n \geq d$): a unique solution

Solving the normal equations for $w^\star$ requires inverting $X^\top X \in \mathbb{R}^{d\times d}$, so before stating the closed-form solution we need to understand when this is possible.

In the underparametrised setting, $X^\top X$ is (nearly always) invertible. From Lemma 2.1, we have that $X^\top X$ is invertible if and only if X has full column rank, i.e. if and only if $\operatorname{span}(x_1, \ldots, x_n) = \mathbb{R}^d$. This means that the inputs must span the whole $\mathbb{R}^d$ space, hence $n \geq d$. In fact, one can show that for "nearly all" datasets for which $n \geq d$, it holds that $\operatorname{rank}(X) = d$ and hence $X^\top X$ is invertible (see Section 2.3 for more details).

Proposition 5.2 (Closed-form solution)**.** *Assuming that* $\operatorname{span}(x_1, \ldots, x_n) = \mathbb{R}^d$*, then* $X^\top X \in \mathbb{R}^{d\times d}$ *is invertible and the least-squares problem has a unique minimiser, given by*

$$w^\star = (X^\top X)^{-1} X^\top y.$$

Geometric interpretation. The vector of predictions Xw ranges, as w varies over $\mathbb{R}^d$, exactly over the column space $\operatorname{span}(\text{columns of } X) \subseteq \mathbb{R}^n$. Minimising $\|Xw - y\|$ therefore means finding the point of this subspace that is *closest* to y: the optimal prediction $Xw^\star$ is the *orthogonal projection* of y onto the column space of X. Equivalently, the residual $Xw^\star - y$ is orthogonal to every column of X, which is precisely the statement of the normal equations $X^\top(Xw^\star - y) = 0$.

Exercise 5.3 ($\star\star$)**.** Let $w^\star$ be any global minimiser of L. Show that L can be rewritten around $w^\star$ as:

$$L(w) = L(w^\star) + \tfrac{1}{2n} \left\|X(w - w^\star)\right\|^2.$$

5.2 Overparametrised setting ($d > n$): an infinite space of solutions

We now consider the overparametrised setting where $d > n$. In this case $X^\top X$ cannot be inverted, so we solve the normal equations differently.

Generic interpolation. In the overparametrised setting where $d > n$ (in fact, all of the following also holds for $d = n$), the inputs $x_1, \ldots, x_n$ can span a linear subspace of $\mathbb{R}^d$ of dimension at most n. Moreover, if they are linearly independent (which holds for almost all datasets, see Section 2.3), then their span is of dimension exactly n, which means that $\operatorname{rank}(X) = n$. This observation leads to the following proposition.

Proposition 5.4 (Zero training error)**.** *If the inputs* $x_1, \ldots, x_n$ *are linearly independent (which requires* $d \geq n$*), then* $\operatorname{rank}(X) = n$ *and the linear map*

$$X : \mathbb{R}^d \to \mathbb{R}^n, \qquad w \mapsto Xw$$

is surjective. Hence for every *response vector* $y \in \mathbb{R}^n$ *there exists at least one weight* $w^\star$ *with* $Xw^\star = y$*. This means that* $y_i = \langle x_i, w^\star \rangle$ *for all* $i \in \{1, \ldots, n\}$*, which means that the training data is fitted perfectly. Also note that this means that* $L(w^\star) = \frac{1}{2n}\|Xw^\star - y\|^2 = 0$*, hence that* $w^\star \in \arg\min L$*.*

The minimisers form an affine space. As soon as one interpolating weight exists the set of global minimisers of L is *exactly* the set of interpolants,

$$S = \{\, w \in \mathbb{R}^d \;:\; Xw = y \,\}.$$

This is an *affine subspace* of $\mathbb{R}^d$. Now let $w^\star$ be any particular solution ($Xw^\star = y$), then any other solution w such that $Xw = y$ is equivalent to $X(w - w^\star) = 0$, which means that $\langle x_i, w - w^\star \rangle = 0$ for every i. Hence $w - w^\star$ is orthogonal to every input, that is, $w - w^\star \in \operatorname{span}(x_1, \ldots, x_n)^\perp$. Hence

$$S = w^\star + \operatorname{span}(x_1, \ldots, x_n)^\perp.$$

Now assuming that the inputs are linearly independent, then the dimension of the affine space S is equal to $d - n$. In other words, there is a whole $(d-n)$-dimensional affine space of weights achieving zero training error.

The minimum-norm solution. Because the whole affine space S is optimal for the training loss, we need an extra criterion to pick a single predictor. A natural and very common choice is the *minimum-norm* interpolant, i.e. the point of S closest to the origin:

$$w^\dagger = \operatorname*{argmin}_{w \,:\, Xw = y} \|w\|.$$

Remark (Why is $w^\dagger$ unique?)**.** *Writing $w^\dagger$ as a single vector (rather than a set of minimisers) implicitly claims that this problem has exactly one solution. This holds because the function $w \mapsto \|w\|^2$ is strictly convex and S is convex (it is an affine space). If $w_1 \neq w_2$ were two minimisers in S with common norm r, then their midpoint $\frac{w_1+w_2}{2}$ would also lie in S and, by strict convexity, satisfy $\|\frac{w_1+w_2}{2}\|^2 < \frac{1}{2}\|w_1\|^2 + \frac{1}{2}\|w_2\|^2 = r^2$, contradicting minimality.*
Geometrically, $w^\dagger$ is the orthogonal projection of the origin onto the affine space S, and the orthogonal projection onto a non-empty closed convex set is always unique. The proposition below gives an explicit formula for it.

Proposition 5.5 (Minimum-norm solution)**.** *If $\operatorname{rank}(X) = n$, then $XX^\top \in \mathbb{R}^{n \times n}$ is invertible and*

$$w^\dagger = X^\top (XX^\top)^{-1} y$$

is the unique minimum-norm solution of $Xw = y$.

Geometric proof. The idea is purely geometric: $w^\dagger$ is the orthogonal projection of the origin onto the affine space of interpolants, and Pythagoras does the rest. Recall that the set of minimisers of L is the affine space

$$S = \{\, w \in \mathbb{R}^d : Xw = y \,\},$$

whose direction is the subspace $\operatorname{span}(x_1, \ldots, x_n)^\perp$.

Step 1: $w^\dagger$ is an interpolant lying in $\operatorname{span}(x_1, \ldots, x_n)$. Since $\operatorname{rank}(X) = n$, the matrix $XX^\top$ is invertible, so $w^\dagger = X^\top (XX^\top)^{-1} y$ is well defined and

$$Xw^\dagger = XX^\top (XX^\top)^{-1} y = y,$$

so $w^\dagger \in S$. Moreover, writing $u = (XX^\top)^{-1}y \in \mathbb{R}^n$, we have $w^\dagger = X^\top u = \sum_{i=1}^n u_i\, x_i$, so $w^\dagger \in \operatorname{span}(x_1, \dots, x_n)$.

Step 2: Pythagoras. Let $w^\star \in S$ be an arbitrary interpolant, i.e. $Xw^\star = y$. Decompose it around $w^\dagger$ as

$$w^\star = w^\dagger + h, \qquad h := w^\star - w^\dagger.$$

Because $Xw^\star = Xw^\dagger = y$, the difference satisfies $Xh = 0$, that is, $h \in \operatorname{span}(x_1, \dots, x_n)^\perp$. But $w^\dagger \in \operatorname{span}(x_1, \dots, x_n)$ by Step 1, so $w^\dagger$ and h are orthogonal, $\langle w^\dagger, h\rangle = 0$. The Pythagorean theorem then gives

$$\|w^\star\|^2 = \|w^\dagger + h\|^2 = \|w^\dagger\|^2 + \|h\|^2 \geq \|w^\dagger\|^2,$$

so $\|w^\star\| \geq \|w^\dagger\|$ for every interpolant $w^\star$. Equality forces $\|h\| = 0$, i.e. $w^\star = w^\dagger$, which proves that $w^\dagger$ is the *unique* minimum-norm interpolant. □

Remark (Two dual regimes)**.** *The formulas $w^\star = (X^\top X)^{-1}X^\top y$ (under-parametrised, $n \geq d$) and $w^\dagger = X^\top(XX^\top)^{-1}y$ (over-parametrised, $d > n$) are mirror images: in the first $X^\top X \in \mathbb{R}^{d\times d}$ is invertible and the solution is unique; in the second $XX^\top \in \mathbb{R}^{n\times n}$ is invertible while $X^\top X$ is singular, so the solution is only unique* after *imposing the minimum-norm rule. Both are special cases of the* Moore-Penrose pseudo-inverse X^+*, for which $w = X^+y$ in either regime.*

Exercise 5.6 ($\star$)**.** In the boundary regime where $n = d$ with $\operatorname{rank}(X) = n = d$, prove that the under-parametrised and over-parametrised formulas match, i.e. that $(X^\top X)^{-1}X^\top y = X^\top(XX^\top)^{-1}y$.

5.3 Gradient descent and stochastic gradient descent

The above equations give a closed-form solution to least squares, but computing them is not always practical, for two reasons.

- **Cost.** In practice one does not compute the inverse $(X^\top X)^{-1}$ (nor $(XX^\top)^{-1}$) explicitly, which is both wasteful and numerically unstable. The normal equations $X^\top X\, w = X^\top y$ form a linear system, and one solves it directly, for instance by a Cholesky factorisation of the symmetric positive definite matrix $X^\top X$, or, better conditioned, by a QR factorisation or an SVD of X that avoids forming $X^\top X$ at all. The cost is nonetheless of order $O(nd^2 + d^3)$. When the number of features d is large, of the order of thousands or millions as in modern models, this becomes prohibitive in both time and memory, whichever method is used.
- **Generality.** The closed form is special to the squared loss with a model that is linear in w. As soon as we change the loss (absolute error, Huber, …) or use a model that is nonlinear in w (a neural network), there is *no* closed-form minimiser at all.

In both cases we give up on solving for w exactly and instead compute an approximate minimiser *iteratively*, using only gradients of L.

Gradient descent. Gradient descent (GD) starts from an arbitrary $w_0 \in \mathbb{R}^d$ and repeatedly takes a small step in the direction of steepest descent, i.e. opposite to the gradient:

$$w_{t+1} = w_t - \gamma \nabla L(w_t), \qquad t = 0, 1, 2, \dots,$$

where $\gamma > 0$ is the *step size* (or *learning rate*). The intuition is that $-\nabla L(w_t)$ points in the direction along which L decreases the most, so for a small enough γ each step lowers the loss. For the least-squares objective the gradient was computed above,

$$\nabla L(w) = \tfrac{1}{n} X^\top (Xw - y),$$

so one GD step costs a matrix-vector product with X and with $X^\top$, that is $O(nd)$ operations, and requires no matrix inversion. Since L is convex, GD with a suitable step size converges to a global minimiser.

> **Remark** (Choosing the step size)**.** *The step size γ must be tuned. If γ is too large the iterates can overshoot the minimum and even diverge; if it is too small the algorithm is stable but crawls, needing very many iterations. For least squares one can be precise, because the Hessian $\nabla^2 L = \frac{1}{n} X^\top X$ is a constant matrix. Write*
>
> $$\beta := \lambda_{\max}\big(\tfrac{1}{n} X^\top X\big) = \tfrac{1}{n} \lambda_{\max}(X^\top X)$$
>
> *for its largest eigenvalue, the* smoothness constant *of L. Then the iterates converge for every step size in the range*
>
> $$0 < \gamma < \frac{2}{\beta},$$
>
> *and diverge as soon as $\gamma > 2/\beta$; this is exactly the threshold appearing in Fact 5.10 below. The usual default is $\gamma = 1/\beta$, which sits comfortably inside the admissible range.*

A variational viewpoint on gradient descent. Gradient descent looks like a purely mechanical rule, "take a step against the gradient". It is enlightening to notice that each step is in fact the solution of a small optimisation problem: at w_t we do not minimise L itself (which we cannot do in closed form), but a cheap *surrogate* obtained by linearising L around w_t and forbidding ourselves from moving too far.

> **Proposition 5.7** (GD as linearise-and-penalise)**.** *For any $w_t \in \mathbb{R}^d$ and step size $\gamma > 0$, the gradient descent update $w_{t+1} = w_t - \gamma \nabla L(w_t)$ is the unique minimiser of the surrogate objective*
>
> $$w_{t+1} = \operatorname*{argmin}_{w \in \mathbb{R}^d} \Big\{ \underbrace{L(w_t) + \langle \nabla L(w_t),\, w - w_t \rangle}_{\text{linearisation of } L \text{ at } w_t} + \frac{1}{2\gamma} \|w - w_t\|^2 \Big\}.$$

Proof. The bracketed function is a strictly convex quadratic in w (the linear part contributes nothing to the curvature, and the penalty $\frac{1}{2\gamma}\|w - w_t\|^2$ has positive-definite Hessian $\frac{1}{\gamma} I$), so it has a unique minimiser, found by cancelling its gradient:

$$\nabla L(w_t) + \frac{1}{\gamma}(w - w_t) = 0 \quad \Longleftrightarrow \quad w = w_t - \gamma \nabla L(w_t). \qquad \square$$

So the step size γ plays the role of a *trust region*: a large γ weakens the proximity penalty and lets the step follow the linear model further, while a small γ keeps w_{t+1} close to w_t, where the linearisation is trustworthy.

Exercise 5.8 ($\star$)**.** What recursion do we obtain if we keep the true loss L instead of its linearisation, i.e. if we define $w_{t+1} = \operatorname{argmin}_w \{L(w) + \frac{1}{2\gamma}\|w - w_t\|^2\}$? (This is known as *implicit* gradient descent, or the proximal point method.)

Stochastic gradient descent. Even though GD avoids matrix inversion, every single step still evaluates the full gradient, which is a sum over all n training points. Writing the empirical risk as a finite sum,

$$L(w) = \frac{1}{n}\sum_{i=1}^{n} \ell_i(w), \qquad \ell_i(w) = \ell(f_w(x_i), y_i),$$

we see that one full gradient $\nabla L(w) = \frac{1}{n}\sum_{i=1}^{n} \nabla \ell_i(w)$ costs $O(nd)$, which is again expensive when the number of samples n is huge.

Stochastic gradient descent (SGD) replaces the full gradient by the gradient at a *single* training point, drawn at random. At each step one samples an index i_t uniformly in $\{1, \dots, n\}$ and updates

$$w_{t+1} = w_t - \gamma_t \nabla \ell_{i_t}(w_t).$$

The key point is that this cheap direction is, on average, the right one:

$$\mathbb{E}_{i_t}\left[\nabla \ell_{i_t}(w_t)\right] = \frac{1}{n}\sum_{i=1}^{n} \nabla \ell_i(w_t) = \nabla L(w_t),$$

so $\nabla \ell_{i_t}(w_t)$ is an *unbiased estimate* of the true gradient, obtained at cost $O(d)$ instead of $O(nd)$. For least squares the per-sample gradient is simply

$$\nabla \ell_i(w) = \left(\langle w, x_i\rangle - y_i\right) x_i.$$

The vocabulary you will meet in practice. Two conventions surround SGD in every library, and they are worth naming.

Mini-batches. Rather than a single sample, one usually averages the gradient over a small *mini-batch* $B_t \subseteq \{1, \dots, n\}$ of $B = |B_t|$ points drawn at random,

$$w_{t+1} = w_t - \gamma_t \frac{1}{B}\sum_{i \in B_t} \nabla \ell_i(w_t).$$

This is still an unbiased estimate of $\nabla L(w_t)$, costs $O(Bd)$, and its variance is smaller by a factor B. The two extremes $B = 1$ and $B = n$ are plain SGD and plain GD, so B interpolates between them, trading variance against cost per step. Typical values are $B = 32$ to 512: large enough for the step to be a meaningful direction, small enough that many steps are taken per pass through the data.

Epochs. One *epoch* is one full pass over the training set, that is n/B steps. Training time is reported in epochs rather than in steps, because it is the number of times each data point has been looked at. In practice the mini-batches of an epoch are usually obtained by shuffling the n indices once and cutting the result into blocks of size B (sampling *without* replacement), which is marginally biased compared with the idealised uniform sampling above, but simpler and slightly more efficient.

Exercise 5.9 ($\star$)**.** What is the analogue of Proposition 5.7 for SGD? That is, which surrogate objective does the SGD update $w_{t+1} = w_t - \gamma \nabla \ell_{i_t}(w_t)$ minimise at each step?

Implicit bias of gradient descent. In the over-parametrised regime $d > n$ there is a whole affine space of minimisers of L, and it is a priori unclear which one will be recovered by gradient descent. To answer this question, the crucial observation is that *every gradient of L lives in the span of the data.* Indeed,

$$\nabla L(w) = \tfrac{1}{n} X^\top (Xw - y) = \tfrac{1}{n} \sum_{i=1}^{n} \left(\langle w, x_i \rangle - y_i\right) x_i \;\in\; \operatorname{span}(x_1, \dots, x_n).$$

Write $V := \operatorname{span}(x_1, \dots, x_n)$ for this subspace.

Throughout, we take for granted the following statement. It is a genuine *theorem*, not a hypothesis on the data: it can be proven, but the proof lies outside the scope of this class, so we simply admit it here.

Fact 5.10 (Convergence of the iterates, admitted)**.** *For the least-squares loss and a small enough step size (any $0 < \gamma < 2/\beta$ with $\beta = \frac{1}{n}\lambda_{\max}(X^\top X)$ the largest eigenvalue of the Hessian), the gradient descent iterates converge to some limit $w_\infty = \lim_{t \to \infty} w_t$.*

Proposition 5.11 (Implicit bias of GD)**.** *Let $(w_t)_{t \geq 0}$ be the gradient descent iterates $w_{t+1} = w_t - \gamma \nabla L(w_t)$ on the least-squares loss L, started at $w_0 \in \mathbb{R}^d$, and let w_∞ be their limit. Then:*

1. *w_∞ is a minimiser of L;*
2. *among* all *minimisers of L, the point w_∞ is the one closest to the starting point w_0, that is, the orthogonal projection of w_0 onto the set of solutions:*

$$w_\infty = \operatorname*{argmin}_{w \in S} \; \|w - w_0\|.$$

In particular, starting from $w_0 = 0$ recovers the minimum-norm solution $w^\dagger$.

Geometric proof. (1) The iterates stay in $w_0 + V$. Each gradient $\nabla L(w_t)$ lies in V by the observation above, so $w_{t+1} - w_t = -\gamma \nabla L(w_t) \in V$. By induction $w_t - w_0 \in V$ for every t, i.e. $w_t \in w_0 + V$. Since V is a closed subspace, the limit inherits the property: $w_\infty \in w_0 + V$.

(2) w_∞ is a minimiser. This is the only place the convergence assumption is used. Passing to the limit in the update $w_{t+1} = w_t - \gamma \nabla L(w_t)$ and using continuity of ∇L, the limit is a fixed point, $w_\infty = w_\infty - \gamma \nabla L(w_\infty)$, which forces $\nabla L(w_\infty) = 0$. As L is convex, this means $w_\infty \in \operatorname{argmin} L =: S$, the set of solutions of the normal equations. Recall that S is an affine space with direction $V^\perp$.

(3) Pythagoras. Let $w^\star \in S$ be any minimiser. Both w_∞ and $w^\star$ lie in S, whose direction is $V^\perp$, so $w_\infty - w^\star \in V^\perp$. On the other hand $w_\infty - w_0 \in V$ by part (1). These two vectors are therefore orthogonal, and Pythagoras gives

$$\|w^\star - w_0\|^2 = \| \underbrace{(w^\star - w_\infty)}_{\in V^\perp} + \underbrace{(w_\infty - w_0)}_{\in V} \|^2 = \|w^\star - w_\infty\|^2 + \|w_\infty - w_0\|^2 \;\geq\; \|w_\infty - w_0\|^2.$$

Thus w_∞ minimises the distance to w_0 over all minimisers, with equality only when $w^\star = w_\infty$: it is the orthogonal projection of w_0 onto S. □

This is what is meant by the *implicit bias* (or *implicit regularisation*) of gradient descent: although the loss L is blind to the choice of interpolant, the optimisation algorithm is not, and it silently selects the solution closest to its initialisation.

Exercise 5.12 ($\star$)**.** What happens for SGD? And what if the loss ℓ is not the square norm, but some other coercive loss?

5.4 Ridge regression (weight decay)

The implicit bias of gradient descent selects a solution *implicitly*, through the dynamics of the algorithm. Ridge regression does the same thing *explicitly*, by adding a penalty to the loss itself that discourages large weights. One augments the least-squares objective with an ℓ_2 penalty (called *weight decay* in the deep-learning community),

$$L_\lambda(w) = \underbrace{\frac{1}{2n}\|Xw - y\|^2}_{L(w)} + \frac{\lambda}{2}\|w\|^2,$$

where $\lambda > 0$ is the *regularisation strength*. The extra term pulls the solution towards the origin; the larger λ, the stronger the pull.

Remark (Two practical warnings: the intercept and the scale of the features)**.** *Unlike everything that came before, the penalty $\|w\|^2$ is* not *invariant under the harmless-looking reparametrisations we have been allowing ourselves, and this has two concrete consequences.*

- Do not penalise the intercept. *Recall that we absorbed an intercept b into w by appending a constant coordinate* 1 *to each input. If we now penalise the whole of $\|w\|^2$, we are also shrinking b towards* 0*, which amounts to saying that a response centred around* 1000 *is somehow less plausible than one centred around* 0*: the fit would change if we merely chose to measure y in different units, or from a different origin. In practice one therefore penalises only the genuine weights, minimising $\frac{1}{2n}\|Xw + b\mathbf{1} - y\|^2 + \frac{\lambda}{2}\|w\|^2$, or equivalently centres y and the columns of X beforehand and drops the intercept altogether.*
- Standardise the features first. *A single penalty λ is applied to every coordinate of w, but the coordinates of x need not be comparable: a surface in m^2 and a surface in km^2 differ by a factor 10^6, and the corresponding weight by a factor 10^{-6}. Ridge would then penalise the two encodings of the* same *model completely differently, whereas ordinary least squares is entirely insensitive to such a rescaling. The remedy, and the other reasons for applying it, are collected in Appendix A.1.*

Below we ignore both points and penalise $\|w\|^2$ as written, which keeps the formulas clean; just remember to centre and standardise before applying them.

Unlike plain least squares, ridge regression always has a *unique* solution, even in the over-parametrised regime $d > n$ where $X^\top X$ is not invertible.

Proposition 5.13 (Existence, uniqueness, closed form)**.** *For every $\lambda > 0$ the objective L_λ is strictly convex and coercive, so it admits a* unique *minimiser*

$$w_\lambda^\star = \left(X^\top X + n\lambda I\right)^{-1} X^\top y,$$

which is well defined for any *$X \in \mathbb{R}^{n\times d}$, whatever the relative sizes of n and d.*

Proof. The Hessian of L_λ is $\nabla^2 L_\lambda = \frac{1}{n}X^\top X + \lambda I \succeq \lambda I \succ 0$, so L_λ is strictly convex. We can therefore find its minimum by finding the point where its gradient vanishes.

$$\nabla L_\lambda(w) = \tfrac{1}{n}X^\top(Xw - y) + \lambda w = 0 \quad \Longleftrightarrow \quad \left(X^\top X + n\lambda I\right) w = X^\top y.$$

Finally $X^\top X + n\lambda I$ is positive definite (its eigenvalues are $\geq n\lambda > 0$), hence invertible, which gives the announced formula. □

Exercise 5.14 ($\star$)**.** Prove that $w_\lambda^\star$ is also equal to:

$$w_\lambda^\star = X^\top (XX^\top + n\lambda I_n)^{-1} y.$$

Deduce that $w_\lambda^\star \in \operatorname{span}(x_1, \ldots, x_n)$. Which of the two formulas would you implement when $d \gg n$?

The two extremes. The parameter λ interpolates between two meaningful limits: crank it up and the penalty dominates, killing the weights; send it to zero and we recover ordinary least squares, more precisely its minimum-norm solution.

Proposition 5.15 (Limits in λ)**.** *Let $w^\dagger$ denote the minimum-norm minimiser of L (the orthogonal projection of 0 onto the affine set $S = \operatorname{argmin} L$). Then*

$$\lim_{\lambda\to+\infty} w_\lambda^\star = 0 \qquad \textit{and} \qquad \lim_{\lambda\to 0^+} w_\lambda^\star = w^\dagger.$$

Proof. Limit $\lambda \to \infty$. Comparing the optimal value with the one at $w = 0$,

$$\tfrac{\lambda}{2}\|w_\lambda^\star\|^2 \leq L_\lambda(w_\lambda^\star) \leq L_\lambda(0) = L(0) = \tfrac{1}{2n}\|y\|^2,$$

so $\|w_\lambda^\star\|^2 \leq \|y\|^2/(n\lambda) \to 0$.

Limit $\lambda \to 0^+$. By optimality of $w_\lambda^\star$ for L_λ, tested against $w^\dagger$,

$$L(w_\lambda^\star) + \tfrac{\lambda}{2}\|w_\lambda^\star\|^2 \;\leq\; L(w^\dagger) + \tfrac{\lambda}{2}\|w^\dagger\|^2. \tag{$*$}$$

Since $w^\dagger$ minimises L, we have $L(w_\lambda^\star) \geq L(w^\dagger)$, and $(*)$ gives $\|w_\lambda^\star\| \leq \|w^\dagger\|$: the family is bounded. Rearranging $(*)$ also gives $0 \leq L(w_\lambda^\star) - L(w^\dagger) \leq \frac{\lambda}{2}\|w^\dagger\|^2 \to 0$, so $L(w_\lambda^\star) \to \min L$. Now let $\bar{w}$ be the limit of any convergent subsequence $w_{\lambda_k}^\star$ (one exists, by boundedness). By continuity $L(\bar{w}) = \min L$, so $\bar{w} \in S$, and $\|\bar{w}\| \leq \|w^\dagger\|$. But $w^\dagger$ is the *unique* element of S of minimal norm, so $\bar{w} = w^\dagger$. Every convergent subsequence thus has the same limit $w^\dagger$, and since the family is bounded, $w_\lambda^\star \to w^\dagger$. □

Remark (Two roads to the same solution)**.** *The limit $\lambda \to 0^+$ is remarkable: it recovers exactly the minimum-norm solution $w^\dagger$ that gradient descent from $w_0 =$*

0 *finds through its implicit bias. Explicit regularisation (ridge, taken to the zero-penalty limit) and implicit regularisation (gradient descent) therefore lead to the* same *predictor. This is one of the cleanest illustrations of why* $w^\dagger$ *is the "natural" least-squares solution in the over-parametrised regime.*

5.5 Feature expansion

A linear model can be a curved function. Models which are linear in the input data x are often too rigid. A flexible remedy is to transform the input using a *chosen* (as opposed to learnt) transformation ϕ. Such a function is often called a *feature map*:

$$\phi : \mathbb{R}^d \to \mathbb{R}^{d'}, \qquad x \mapsto \phi(x) = (\phi_1(x), \dots, \phi_{d'}(x)),$$

which sends the raw input, living in its native dimension d, to a vector of d' new features. After this *feature augmentation*, one wants to learn a linear model of these features:

$$f_w(x) = \langle w, \phi(x) \rangle, \qquad w \in \mathbb{R}^{d'}.$$

The number of features d' is ours to choose, and it may be larger, sometimes much larger, than the raw input dimension d.

Example 5.16 (Polynomial features)**.** *For a scalar input* $z \in \mathbb{R}$ *(so* $d = 1$*), the feature map* $\phi(z) = (1, z, z^2, \dots, z^k) \in \mathbb{R}^{k+1}$ *turns the model into a degree-k polynomial*

$$f_w(z) = w_0 + w_1 z + \cdots + w_k z^k,$$

built out of $d' = k + 1$ *features. The predictor is now a* nonlinear *function of* z*, yet it remains a* linear *function of the parameter* w*.*

Everything happens as if the data had been transformed once and for all. Transform each input once, before doing anything else, and call the result $\tilde{x}_i = \phi(x_i) \in \mathbb{R}^{d'}$. We are then fitting the plain linear model $\tilde{x} \mapsto \langle w, \tilde{x} \rangle$ on the dataset $(\tilde{x}_1, y_1), \dots, (\tilde{x}_n, y_n)$, whose design matrix stacks the *transformed* inputs:

$$\tilde{X} = \begin{pmatrix} - & \tilde{x}_1 & - \\ & \vdots & \\ - & \tilde{x}_n & - \end{pmatrix} = \begin{pmatrix} - & \phi(x_1) & - \\ & \vdots & \\ - & \phi(x_n) & - \end{pmatrix} \in \mathbb{R}^{n \times d'}, \qquad y = (y_1, \dots, y_n) \in \mathbb{R}^n.$$

The empirical risk is then

$$L(w) = \frac{1}{2n} \sum_{i=1}^{n} \big(\langle w, \phi(x_i) \rangle - y_i \big)^2 = \frac{1}{2n} \sum_{i=1}^{n} \big(\langle w, \tilde{x}_i \rangle - y_i \big)^2 = \frac{1}{2n} \| \tilde{X} w - y \|^2,$$

which is word for word the least-squares problem of Section 5, with $\tilde{X}$ in place of X and d' in place of d. Normal equations, closed-form solution, minimum-norm interpolant, gradient descent and its implicit bias, ridge regression: not one of these formulas ever cares whether the rows of the design matrix were transformed beforehand.

Under and over-parametrisation with polynomial expansion. Feature augmentation lets us increase d' freely, and polynomial features are the cleanest way to see what happens as d' crosses n, the number of training points. For a scalar input $z \in \mathbb{R}$, write $\phi^{(k)}$ for the feature map of degree k,

$$\phi^{(0)}(z) = (1), \quad \phi^{(1)}(z) = \begin{pmatrix} 1 \\ z \end{pmatrix}, \quad \phi^{(2)}(z) = \begin{pmatrix} 1 \\ z \\ z^2 \end{pmatrix}, \quad \dots, \quad \phi^{(k)}(z) = \begin{pmatrix} 1 \\ z \\ \vdots \\ z^k \end{pmatrix} \in \mathbb{R}^{k+1}.$$

A degree-k polynomial model therefore works in dimension $d' = k + 1$, and nothing stops us from taking d' as large as we like. Given n training points $(z_1, y_1), \dots, (z_n, y_n)$ with distinct abscissae $z_i \in \mathbb{R}$, three regimes appear:

- *(Strictly) under-parametrised setting $d' < n$ (polynomials of degree less than $n - 1$)*: it is highly unlikely to be able to exactly pass through n points with a polynomial of degree less than $n - 1$. There exists a unique polynomial which minimises the square error.
- *Boundary $d' = n$ (polynomials of degree $n - 1$)*: there exists a unique polynomial of degree $(n - 1)$ passing through all of the points exactly (Lagrange interpolation). That is, a weight vector $w \in \mathbb{R}^n$ with $f_w(x_i) = y_i$ for every i.
- *Over-parametrised setting $d' > n$ (polynomials of degree $\geq n$)*: there exists an infinite number of ways of interpolating the points.

> **Remark** (What "linear regression" really means)**.** *"Linear regression" refers to models that are* linear in the parameters w, not *necessarily linear in the input x. Both $f_w(x) = \langle w, x \rangle$ and the polynomial model $f_w(x) = \langle w, \phi(x) \rangle$ above are linear regression, even though the latter is a curved function of x. This is exactly the property that makes the least-squares objective a convex quadratic in w, and it is why everything proved so far survives untouched, as we checked above.*

Learnt features: back to neural networks. Feature expansion also gives an interesting way of seeing the neural network we met in Section 4.2. Grouping its hidden units into a vector, one can write

$$f_w(x) = \sum_{k=1}^{m} a_k \, \sigma\big(\langle b_k, x \rangle + c_k\big) = \langle a, \phi_{b,c}(x) \rangle, \qquad \phi_{b,c}(x) = \Big(\sigma\big(\langle b_k, x \rangle + c_k\big)\Big)_{1 \leq k \leq m},$$

so that a neural network is nothing but a linear model on top of a feature map: the m hidden neurons are the features, and the output weights $a \in \mathbb{R}^m$ play the role of w. If the inner parameters (b_k, c_k) are frozen, for instance drawn at random once and never touched again, then $\phi_{b,c}$ is a fixed feature map and fitting a is exactly the least-squares problem of this section, closed-form solution included. This is the *random features* model.

The whole difference is that a neural network does not freeze them: it trains the (b_k, c_k) together with a. The features are then no longer picked by hand beforehand, they are *learnt from the data*, which is what makes these models so effective on inputs (images, sound, text) for which nobody knows how to design a good ϕ. The price is the one announced earlier: the model is no longer linear in its parameters, the objective is no longer convex, and there is no closed-form solution.

6 Classification

We now move from predicting a real number to predicting a *category*. As explained at the beginning of these notes, the general idea does not change: we pick a class of functions, measure the quality of a prediction with an appropriate loss, and minimise the empirical risk. What changes is the nature of y, and therefore the loss. Almost everything we built for regression, and in particular gradient descent and regularisation, carries over untouched.

6.1 Binary classification

We start with two categories. The data is as before,

$$(x_1, y_1), \dots, (x_n, y_n) \in \mathbb{R}^d \times \mathcal{Y},$$

but now the label lives in a set of two elements. It will be very convenient to take

$$\mathcal{Y} = \{-1, +1\},$$

and we keep this convention throughout the section. Think of $+1$ as "the email is spam" and -1 as "it is not".

Linear classifiers. We keep the linear model $\langle w, x\rangle$, but its output is a real number, whereas we now need to produce a sign. The natural fix is to threshold it: the predictor associated with $w \in \mathbb{R}^d$ is

$$f_w(x) = \operatorname{sign}\big(\langle w, x\rangle\big) \in \{-1, +1\}.$$

The quantity $\langle w, x\rangle$ is called the *score* of x. Geometrically, the set where the score vanishes,

$$H_w = \{\, x \in \mathbb{R}^d \;:\; \langle w, x\rangle = 0 \,\},$$

is a hyperplane through the origin (with the intercept trick below, an arbitrary hyperplane), and f_w answers according to the side of H_w on which x falls. The hyperplane H_w is called the *decision boundary*, and w is orthogonal to it.

> **Remark** (The intercept, again)**.** *Exactly as in Section 5.4 and before it, an affine decision boundary $\langle w, x\rangle + b = 0$ is obtained by appending a constant coordinate 1 to each input and an intercept b to the weights. We therefore keep writing $\langle w, x\rangle$ without loss of generality.*

The margin. A key quantity of this section is the product of the label and the score.

> **Definition** (Margin)**.** The *margin* of the predictor w on the sample (x_i, y_i) is
>
> $$m_i(w) \coloneqq y_i \,\langle w, x_i\rangle \;\in \mathbb{R}.$$

The margin packages the correctness of a prediction into a single signed number:

- $m_i > 0$ means $\langle w, x_i\rangle$ and y_i have the same sign, i.e. the point is *correctly classified*;
- $m_i < 0$ means the point is *misclassified*;

- $|m_i|/\|w\|$ is the distance of the point to the decision boundary (see the exercise below), so for a fixed $\|w\|$ a large positive margin means "correct, and confidently so".

This is precisely why the labels ± 1 are pleasant: they let a single expression $y_i\langle w, x_i\rangle$ carry both the prediction and whether it was right. Every loss we meet below is a function of the margin alone.

Exercise 6.1 ($\star$)**.** Prove that $\left|y_i\langle \frac{w}{\|w\|}, x_i\rangle\right| = \min_{x \in H_w} \|x - x_i\|$, i.e. that the normalised margin is the distance of x_i to H_w.

Remark (Labels in $\{0,1\}$)**.** *Many references, and most software, encode the two classes as $\tilde{y} \in \{0,1\}$ rather than $y \in \{-1,+1\}$. Nothing of substance changes: the two conventions are exchanged by the affine bijection*

$$\tilde{y} = \frac{1+y}{2} \in \{0,1\}, \qquad \textit{equivalently} \qquad y = 2\tilde{y} - 1 \in \{-1,+1\}.$$

The $\{0,1\}$ convention is the natural one when the label is read as an indicator *of membership in class 1, which is the form in which most software expects it; the $\{-1,+1\}$ convention is the natural one when the label is read as a* sign*, which is what makes the margin, and hence this whole section, work. We will use ± 1 throughout and translate when useful.*

6.2 Choosing a loss: from the ideal one to convex surrogates

What we would like to minimise. The quantity we actually care about is the proportion of mistakes. This corresponds to the 0/1 *loss*, which we write directly as a function of the margin m:

$$\ell_{0/1}(m) = \mathbf{1}\{m \le 0\} = \begin{cases} 1 & \text{if the point is misclassified,} \\ 0 & \text{otherwise,} \end{cases}$$

whose empirical risk

$$L_{0/1}(w) = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\{y_i\langle w, x_i\rangle \le 0\}$$

is exactly the *misclassification rate* on the training set. Unfortunately it is hopeless to minimise directly. Since it is piecewise constant, we cannot use gradient methods, its gradient is zero wherever it is defined. Gradient descent, started anywhere, does not move. Minimising it is in fact NP-hard in general.

Convex surrogates. The standard remedy is to replace $\ell_{0/1}$ by a *surrogate loss*: a function $\ell(m)$ of the margin that is convex and dominates the 0/1 loss, possibly after multiplication by a positive constant, so that making it small forces the misclassification rate to be small too. Here are the classical choices.

- *Square loss*, $\ell(m) = (1-m)^2$. This is nothing but least squares applied to the labels ± 1, and it is the one loss of the list with a closed form. Its defect is that it is increasing again for $m > 1$: it *penalises points that are classified too well*, which is not what we want.

- *Hinge loss*, $\ell(m) = \max(0, 1 - m)$. It asks each point to be correctly classified with a margin of at least 1, and is perfectly happy beyond that. Combined with an ℓ_2 penalty it gives the *support vector machine*. It is convex but not differentiable at $m = 1$.
- *Logistic loss*, $\ell(m) = \ln\left(1 + e^{-m}\right)$. Convex, smooth, strictly decreasing, and never exactly zero. This is the one we develop below.
- *Exponential loss*, $\ell(m) = e^{-m}$. Convex and smooth, but it penalises misclassified points enormously, which makes it sensitive to outliers. It is the loss underlying boosting.

All of them are decreasing functions of the margin (but the square loss only for $m \leq 1$, which is precisely its problem), and this is the essential point: they all push each m_i to be as large as possible, that is, they all reward classifying correctly and confidently.

6.3 Logistic regression

Why this surrogate and not another. We focus here on the *logistic loss*

$$\ell(m) = \ln\left(1 + e^{-m}\right).$$

This is by far the most used surrogate in practice and it is what essentially every classification neural network is trained on. It combines all the nice following properties:

- It is *convex*, so the empirical risk has no spurious local minima, which is the very point of replacing $\ell_{0/1}$.
- It is *strictly decreasing*, so a point is never penalised for being classified *too* well.
- It grows only *linearly* as $m \to -\infty$, since $\ln(1 + e^{-m}) = -m + \ln(1 + e^{m}) \approx -m$. A single badly misclassified point therefore contributes a bounded slope and cannot hijack the whole objective. The exponential loss fails this badly: e^{-m} blows up, and one outlier can dominate the sum.

Note also its behaviour on the other side: $\ell(m) \to 0$ as $m \to +\infty$ but $\ell(m) > 0$ for every finite m. This means that contrary to the square loss, ℓ does not have a finite minimiser over $\mathbb{R}$.

The logistic risk. Plugging the linear score into the empirical risk with the logistic loss, learning amounts to minimising over $w \in \mathbb{R}^d$ the function

$$L(w) = \frac{1}{n}\sum_{i=1}^{n} \ell(m_i(w)) = \frac{1}{n}\sum_{i=1}^{n} \ln\Big(1 + \exp\big(-y_i\langle w, x_i\rangle\big)\Big).$$

This is the objective of *logistic regression*, and it is the one we minimise for the rest of this section. Two features of it are already visible from the loss alone. Being an average of convex functions of w, it is convex, so we are again in the comfortable situation of Section 5. But every term is strictly positive, so $L(w) > 0$ for every w, and the infimum of L may not be attained.

A convenient notation. Differentiating ℓ produces the same expression over and over, so we give it a name. Let

$$\sigma(m) = \frac{1}{1+e^{-m}}, \qquad m \in \mathbb{R},$$

called the *sigmoid* (or logistic) function. It increases from 0 to 1, with $\sigma(0) = 1/2$, and satisfies the two identities

$$\sigma(-m) = 1 - \sigma(m), \qquad \sigma'(m) = \sigma(m)(1-\sigma(m)) = \sigma(m)\,\sigma(-m).$$

It is related to the loss by $\ell(m) = -\ln \sigma(m)$, and:

$$\ell'(m) = -\sigma(-m) \;\in\; (-1, 0).$$

So $\sigma(-m)$ is exactly the *steepness* of the logistic loss at margin m: close to 1 when $m \ll 0$, close to 0 when $m \gg 0$.

Remark (The same objective with $\{0,1\}$ labels)**.** *With the convention $\tilde{y} \in \{0,1\}$ introduced in Section 6.1, and writing $\sigma_i = \sigma(\langle w, x_i\rangle)$ for the predicted probability of class 1, the formula found in textbooks and libraries is*

$$L(w) = -\frac{1}{n}\sum_{i=1}^{n}\Big[\tilde{y}_i \ln \sigma_i + (1-\tilde{y}_i)\ln(1-\sigma_i)\Big],$$

usually called the binary cross-entropy*. It is the logistic risk in disguise. Since $\tilde{y}_i$ is either 0 or 1, the bracket is a switch: it keeps $\ln \sigma_i = \ln \sigma(\langle w, x_i\rangle)$ when $\tilde{y}_i = 1$, that is when $y_i = +1$, and it keeps $\ln(1-\sigma_i) = \ln \sigma(-\langle w, x_i\rangle)$ when $\tilde{y}_i = 0$, that is when $y_i = -1$. In both cases what survives is $\ln \sigma\big(y_i\langle w, x_i\rangle\big) = \ln \sigma(m_i)$, and the minus sign in front of the sum turns it into $\ell(m_i) = -\ln \sigma(m_i)$. So this is exactly the objective above.*

Exercise 6.2 ($\star$)**.** Prove the last claim: substituting $\tilde{y}_i = (1+y_i)/2$ and $\sigma_i = \sigma(\langle w, x_i\rangle)$ into the cross-entropy formula gives back $\frac{1}{n}\sum_i \ln(1+e^{-y_i\langle w, x_i\rangle})$.

Gradient and Hessian. Since we will have to minimise L by an iterative method, we need its gradient. Write $m_i = m_i(w) = y_i\langle w, x_i\rangle$ for readability.

Proposition 6.3 (Derivatives of the logistic risk)**.** *The function L is differentiable on $\mathbb{R}^d$, with*

$$\nabla L(w) = -\frac{1}{n}\sum_{i=1}^{n}\sigma(-m_i)\, y_i\, x_i = -\frac{1}{n}\sum_{i=1}^{n}\frac{y_i\, x_i}{1+e^{y_i\langle x_i, w\rangle}},$$

and

$$\nabla^2 L(w) = \frac{1}{n}\sum_{i=1}^{n}\sigma(m_i)\,\sigma(-m_i)\, x_i x_i^\top = \frac{1}{n}X^\top D_w X \;\succeq\; 0,$$

where $D_w = \operatorname{diag}\big(\sigma(m_i)\sigma(-m_i)\big)_{1\le i\le n} \in \mathbb{R}^{n\times n}$ is diagonal with positive entries. In particular L is convex. If moreover $\operatorname{rank}(X) = d$, then $\nabla^2 L(w) \succ 0$ for every w and L is strictly convex.

Proof. The scalar function $\ell(m) = \ln(1 + e^{-m})$ has derivative

$$\ell'(m) = \frac{-e^{-m}}{1 + e^{-m}} = \frac{-1}{1 + e^{m}} = -\sigma(-m),$$

and the chain rule applied to $w \mapsto m_i(w) = y_i\langle w, x_i\rangle$, whose gradient is $y_i x_i$, gives the announced ∇L. Differentiating once more,

$$\nabla_w[\sigma(-m_i)] = \sigma'(-m_i) \cdot (-y_i x_i) = -\sigma(-m_i)\sigma(m_i)\, y_i x_i,$$

so that $\nabla^2 L(w) = \frac{1}{n}\sum_i \sigma(m_i)\sigma(-m_i)\, y_i^2\, x_i x_i^\top$, and $y_i^2 = 1$ because $y_i \in \{-1, +1\}$. Each coefficient $\sigma(m_i)\sigma(-m_i)$ is > 0, so for any $u \in \mathbb{R}^d$,

$$u^\top \nabla^2 L(w) u = \frac{1}{n}\sum_{i=1}^{n} \sigma(m_i)\sigma(-m_i)\, \langle u, x_i\rangle^2 \geq 0,$$

which is the convexity. If $\operatorname{rank}(X) = d$ then $\operatorname{Ker}(X) = \{0\}$, so $u \neq 0$ forces $\langle u, x_i\rangle \neq 0$ for at least one i, and the sum is > 0. □

Remark (Reading the gradient)**.** *The formula for ∇L is worth staring at. A gradient* descent *step moves w along $-\nabla L(w)$, that is, along*

$$\frac{1}{n}\sum_{i=1}^{n} \underbrace{\sigma(-m_i)}_{\text{weight} \in (0,1)} \quad \underbrace{y_i x_i}_{\text{direction}} .$$

Each sample pushes w in the direction $y_i x_i$, which is exactly the direction that increases its own margin $m_i = y_i\langle w, x_i\rangle$: every point asks to be classified better. What differs between points is the weight they are given, and that weight is the steepness $\sigma(-m_i) = -\ell'(m_i)$ met above. Badly classified points ($m_i \ll 0$) carry a weight close to 1 and dominate the step; comfortably classified points ($m_i \gg 0$) carry a weight close to 0 and are almost ignored. Logistic regression therefore spends nearly all its effort on the points near the decision boundary or on the wrong side of it, and essentially none on the easy ones.

No closed form. Here is the essential practical difference with least squares. There, setting $\nabla L(w) = 0$ gave the *linear* system $X^\top X w = X^\top y$, which we could solve in closed form. Here, $\nabla L(w) = 0$ reads

$$\sum_{i=1}^{n} \frac{y_i\, x_i}{1 + e^{y_i\langle w, x_i\rangle}} = 0,$$

in which the unknown w appears inside an exponential. This is a system of d transcendental equations, and it has *no closed-form solution*: there is no analogue of the normal equations, and no formula such as $(X^\top X)^{-1} X^\top y$ to write down. We are therefore in the second of the two situations listed at the beginning of Section 5.3, and we do the only thing we can: we minimise L *iteratively*. Concretely,

$$w_{t+1} = w_t - \gamma\, \nabla L(w_t) \qquad \text{or} \qquad w_{t+1} = w_t - \gamma\, \nabla \ell_{i_t}(w_t),$$

that is, exactly the gradient descent and stochastic gradient descent of Section 5.3, with the gradient of Proposition 6.3 plugged in. Since L is convex and smooth, gradient descent with a small enough step size will minimise the training loss.

6.4 Separable and non-separable data

Definition (Linearly separable data)**.** The training set is said to be *linearly separable* if there exists $w \in \mathbb{R}^d$ such that

$$y_i \langle w, x_i \rangle > 0 \qquad \text{for every } i = 1, \dots, n,$$

that is, if some hyperplane classifies every training point correctly. It is said to be *non-separable* if for every $w \neq 0$ there exists i such that

$$y_i \langle w, x_i \rangle < 0,$$

that is, if every candidate direction *strictly* misclassifies at least one point.

Remark (An in-between case)**.** *The two notions are not exact complements: there are datasets which are neither separable nor non-separable in the above sense, namely when no w achieves $y_i \langle w, x_i \rangle > 0$ for all i, but some $w \neq 0$ achieves $y_i \langle w, x_i \rangle \geq 0$ for all i (some points then lie exactly on the decision boundary). This degenerate case has probability zero for data drawn from a density, and we leave it aside.*

Whether or not the data is separable completely changes the behaviour of L.

Separable data: the minimum is at infinity.

Proposition 6.4 (No minimiser in the separable case)**.** *Suppose the data is linearly separable. Then*

$$\inf_{w \in \mathbb{R}^d} L(w) = 0,$$

but the infimum is not attained*: $L(w) > 0$ for every $w \in \mathbb{R}^d$. Moreover every minimising sequence (w_k) (i.e. such that $L(w_k) \to 0$) satisfies $\|w_k\| \to \infty$.*

Proof. Each term $\ln(1 + e^{-m})$ is strictly positive for every finite m, so $L(w) > 0$ for every w, and in particular $\inf L \geq 0$. To show that this infimum is in fact equal to 0, let u be a separating direction and set $\rho := \min_i y_i \langle u, x_i \rangle > 0$, the smallest margin it achieves. Walking along the ray tu with $t > 0$, every margin is multiplied by t:

$$L(tu) = \frac{1}{n} \sum_{i=1}^{n} \ln\left(1 + e^{-t\, y_i \langle u, x_i \rangle}\right) \;\leq\; \ln\left(1 + e^{-t\rho}\right) \;\xrightarrow[t \to +\infty]{}\; 0,$$

so $\inf L \leq 0$, and hence $\inf L = 0$. Since $L(w) > 0$ for every w, this value 0 is never reached: the infimum is not attained. Finally, if some minimising sequence had a bounded subsequence, it would have a convergent one, $w_{k_j} \to \bar{w}$, and continuity of L would give $L(\bar{w}) = 0$, which we have just seen is impossible. Hence $\|w_k\| \to \infty$. □

The picture to keep in mind is this: once a separating direction u is found, scaling it up multiplies all the margins by t without ever changing a single prediction, since $\operatorname{sign}(\langle tu, x \rangle) = \operatorname{sign}(\langle u, x \rangle)$. The classifier is already perfect, but the loss keeps rewarding more confidence, and it can always buy more by making $\|w\|$ larger. So the optimisation problem has no solution: *the minimum sits at infinity*, and gradient descent, faithfully, sends $\|w_t\|$ to infinity. The training loss decreases to 0 while the training accuracy has been 100% for a long time.

Remark (Which direction does it diverge in?)**.** *If $\|w_t\| \to \infty$, the only thing that still matters is the* direction *$w_t/\|w_t\|$, since that is what determines the predictions. And the direction does converge. This is the exact analogue, for the logistic loss, of the implicit bias of gradient descent we proved in Section 5.3 for least squares: the loss alone does not pick a separating hyperplane, but the algorithm does. We record the result without proof.*

Fact 6.5 (Implicit bias of GD on separable data, admitted)**.** *On linearly separable data, gradient descent on the logistic loss with a small enough constant step size satisfies $\|w_t\| \to \infty$, while*

$$\frac{w_t}{\|w_t\|} \xrightarrow[t\to\infty]{} \frac{\hat{u}}{\|\hat{u}\|}, \qquad \hat{u} \in \operatorname*{argmax}_{\|u\|\le 1} \min_{1\le i\le n} y_i\langle u, x_i\rangle.$$

That is, the direction converges to the maximum-margin *separator, the one staying as far as possible from the closest training points. This is the solution of the hard-margin support vector machine, obtained here for free, without ever writing that problem down.*

Non-separable data: a minimum exists. When the two classes overlap, no w can classify everything correctly, and the loss can no longer be driven to 0.

Proposition 6.6 (Existence of a minimiser)**.** *Suppose the data is non-separable, i.e. for every $w \neq 0$ there exists a datapoint such that $y_i\langle w, x_i\rangle < 0$. Then L is coercive and attains its minimum.*

Proof. The function $h(u) := \min_i y_i\langle u, x_i\rangle$ is continuous (a minimum of finitely many linear functions) and, by assumption, $h(u) < 0$ on the unit sphere, which is compact. Hence $\rho := -\max_{\|u\|=1} h(u) > 0$: every unit direction misclassifies some point with margin at most $-\rho$.

Now take any $w \neq 0$ and write $w = tu$ with $t = \|w\|$ and $\|u\| = 1$. Picking an index i with $y_i\langle u, x_i\rangle \le -\rho$ and keeping only that term,

$$L(w) \;\ge\; \frac{1}{n}\ln\left(1 + e^{t\rho}\right) \;\ge\; \frac{t\rho}{n} \xrightarrow[\|w\|\to\infty]{} \infty,$$

so L is coercive. Since L is also continuous, it attains its minimum by Proposition 2.5. □

Weight regularisation. Just as ridge regression cured the non-uniqueness of least squares in the over-parametrised regime, adding an ℓ_2 penalty cures the "minimum at infinity" problem here. Consider

$$L_\lambda(w) = \frac{1}{n}\sum_{i=1}^{n} \ln\left(1 + e^{-y_i\langle w, x_i\rangle}\right) \;+\; \frac{\lambda}{2}\|w\|^2, \qquad \lambda > 0.$$

Being a sum of a convex function and a strictly convex one, L_λ is strictly convex; and $L_\lambda(w) \ge \frac{\lambda}{2}\|w\|^2 \to \infty$, so it is coercive. Being continuous and coercive, it admits a minimiser (Proposition 2.5), and being strictly convex, this minimiser is unique (Section 2.5). So L_λ has a *unique* minimiser, whatever the data, separable or not, and whatever the

relative sizes of n and d. The penalty is precisely what forbids the escape to infinity: buying more confidence now costs $\frac{\lambda}{2}\|w\|^2$, and at some finite point the purchase is no longer worth it. As before, there is still no closed form, and one minimises L_λ by gradient descent, whose gradient is $\nabla L(w) + \lambda w$.

6.5 Multiclass classification

We finally treat $K \geq 2$ categories, so that

$$y_i \in \{1, 2, \ldots, K\},$$

for instance $K = 10$ when recognising which digit is written in an image.

One score per class. A single score is no longer enough: we compute *one score per class*, using one weight vector per class. The parameter is now a collection

$$W = (w_1, \ldots, w_K), \qquad w_k \in \mathbb{R}^d,$$

which we may equally see as a matrix $W \in \mathbb{R}^{d \times K}$, and the scores of an input x are the K numbers $\langle w_1, x\rangle, \ldots, \langle w_K, x\rangle$. The prediction is the class with the largest score,

$$f_W(x) = \operatorname*{argmax}_{1 \leq k \leq K} \langle w_k, x\rangle.$$

The multiclass margin. The reasoning of Section 6.2 repeats verbatim, and again it is the 0/1 loss that we would like to minimise and cannot. Writing $s_k = \langle w_k, x_i\rangle$ for the scores of the sample i, the prediction is correct exactly when the true class wins, that is, when $s_{y_i} > s_k$ for every $k \neq y_i$, equivalently when

$$\max_{k \neq y_i} \; (s_k - s_{y_i}) \; < \; 0.$$

This quantity plays the role that $-m_i$ played in the binary case: it is negative exactly when the point is correctly classified, and the 0/1 loss is the indicator that it is non-negative. It is once more non-convex and flat, so once more we look for a convex surrogate.

Softening the maximum. The obstruction is the maximum, which is convex but not smooth. The standard fix is to replace it by the *log-sum-exp* function, which is a smooth approximation of the maximum: for any reals $a_1, \ldots, a_K$,

$$\max_k a_k \;\leq\; \ln \sum_{k=1}^{K} e^{a_k} \;\leq\; \max_k a_k + \ln K.$$

The left inequality holds because the sum dominates its largest term, the right one because the sum is at most K times its largest term. So log-sum-exp is the maximum, up to an additive constant of at most $\ln K$, and it is convex and C^∞. Applying it to $a_k = s_k - s_{y_i}$ (which now includes the index $k = y_i$, contributing $e^0 = 1$) gives the loss

$$\ell_i(W) = \ln \sum_{k=1}^{K} \exp\left(s_k - s_{y_i}\right) = \ln\Big(1 + \sum_{k \neq y_i} \exp\left(s_k - s_{y_i}\right)\Big),$$

and the empirical risk, after factoring $e^{-s_{y_i}}$ out of the sum:

$$\boxed{L(W) = \frac{1}{n}\sum_{i=1}^{n}\left[-\langle w_{y_i}, x_i\rangle + \ln\sum_{k=1}^{K}\exp\left(\langle w_k, x_i\rangle\right)\right].}$$

This is the *cross-entropy* (or *softmax*) loss. The first term pushes up the score of the correct class, the second pushes down the scores of all classes; their balance is what makes the correct class win. The middle expression makes the kinship with the binary case plain: it is $\ln(1+\sum_{k\neq y} e^{s_k - s_y})$, against $\ln(1+e^{-m})$ for two classes. This is by some distance the most used loss in modern machine learning: it is the objective at the end of essentially every classification network. Evaluating it on a computer requires a little care, since the exponentials overflow easily; see Appendix A.2.

The softmax weights. Differentiating log-sum-exp produces normalised exponentials, which we name once and for all:

$$p_k(x) \coloneqq \frac{\exp\left(\langle w_k, x\rangle\right)}{\sum_{k'=1}^{K}\exp\left(\langle w_{k'}, x\rangle\right)}, \qquad k = 1, \dots, K,$$

called the *softmax weights*. They are positive, they sum to 1, and the largest score gets the largest weight. For $K = 2$ they reduce to sigmoids: $p_1(x) = \sigma(\langle w_1 - w_2, x\rangle)$ and $p_2(x) = \sigma(-\langle w_1 - w_2, x\rangle)$. In particular, the steepness $\sigma(-m_i)$ of the binary case corresponds to $1 - p_{y_i}(x_i)$, the weight the softmax does *not* put on the true class.

Proposition 6.7 (Gradient of the cross-entropy)**.** *For each $k \in \{1, \dots, K\}$,*

$$\nabla_{w_k} L(W) = \frac{1}{n}\sum_{i=1}^{n}\Big(p_k(x_i) - \mathbf{1}\{y_i = k\}\Big)\, x_i.$$

Moreover L is convex in W.

The factor $p_k(x_i) - \mathbf{1}\{y_i = k\}$ compares the weight the softmax currently puts on class k with the weight we would like it to put there, namely 1 for the true class and 0 for all the others. If the sample is already classified with a comfortable margin, the softmax weights are already close to that target, the factor is nearly zero and the sample contributes almost nothing to the step; if it is confidently misclassified (the softmax puts almost all its weight on a wrong class), the factor is close to ± 1 and the sample pushes hard. This is the exact multiclass counterpart of the reading we gave of the binary gradient. Exactly as in the binary case, too, setting all the $\nabla_{w_k} L$ to zero gives a system of transcendental equations with no closed-form solution, and we minimise by gradient descent.

Exercise 6.8 ($\star\star$)**.** Prove Proposition 6.7. Start by showing that the gradient of the second term, $\nabla_{w_k} \ln\sum_{k'} e^{\langle w_{k'}, x\rangle}$, is equal to $p_k(x)\, x$, and then differentiate the first term, which is nonzero only when $k = y_i$.

Remark (The parametrisation is redundant)**.** *Adding the same vector $v \in \mathbb{R}^d$ to* every *w_k leaves the softmax unchanged, since $e^{\langle w_k + v, x\rangle} = e^{\langle v, x\rangle} e^{\langle w_k, x\rangle}$ and the common factor cancels in the ratio. So $L(W)$ is constant along all such directions, and*

a minimiser is never *unique: as in the over-parametrised least-squares problem of Section 5.2, the set of minimisers contains a whole subspace of directions the loss cannot see. One fixes this either by regularising (adding $\frac{\lambda}{2}\sum_k \|w_k\|^2$, which selects the representative of smallest norm) or by fixing $w_K = 0$ by convention.*

Recovering binary classification. Take $K = 2$, with class 1 standing for the label $y = +1$ and class 2 for $y = -1$. The middle form of the loss makes the computation immediate: with only one rival class, the inner sum has a single term, and the loss of a sample of class 1 is

$$\ell_i = \ln\left(1 + e^{\,s_2 - s_1}\right) = \ln\left(1 + e^{-\langle w_1 - w_2,\, x_i\rangle}\right),$$

while for a sample of class 2 the roles of the indices are swapped and

$$\ell_i = \ln\left(1 + e^{\,s_1 - s_2}\right) = \ln\left(1 + e^{+\langle w_1 - w_2,\, x_i\rangle}\right).$$

The two weight vectors therefore enter only through their *difference*, which is exactly the redundancy of the previous remark: the loss is constant along the d-dimensional subspace $\{(v, v) : v \in \mathbb{R}^d\}$ of the parameter space $\mathbb{R}^{2d}$, and what survives is the d-dimensional quotient, coordinatised by $w_1 - w_2$. Setting

$$w \coloneqq w_1 - w_2 \in \mathbb{R}^d,$$

both cases become the single expression $\ln(1 + e^{-y_i\langle w, x_i\rangle})$, so the multiclass cross-entropy with $K = 2$ *is* the logistic risk of Section 6.3, written in the variable $w = w_1 - w_2$. Binary logistic regression is thus not a separate method but the special case $K = 2$ of softmax regression, and the ± 1 margin formulation is what you obtain after removing the redundant direction.

Exercise 6.9 ($\star$)**.** Check the analogous statement for the predictions rather than the loss: verify that $\operatorname{argmax}_{k\in\{1,2\}}\langle w_k, x\rangle = 1$ if and only if $\langle w, x\rangle > 0$, so that the multiclass rule reduces to $\operatorname{sign}(\langle w, x\rangle)$.

7 Unsupervised learning

Everything so far has been supervised: every input x_i came with a label y_i, and the whole machinery was built to predict y from x. We now remove the labels.

7.1 The setup, and what changes without labels

The data is a plain collection of points

$$x_1, \dots, x_n \in \mathbb{R}^d,$$

with nothing attached to them, and we stack them as always into the design matrix $X \in \mathbb{R}^{n \times d}$ whose i-th row is x_i. There is no y, and therefore no function f to fit and no prediction to be scored. The question becomes the vaguer one of *finding structure*: is the cloud of points really spread over all d directions, or does it essentially live in a much smaller subspace? Does it break up into a few well-separated groups?

What we keep: one recipe, two instances. The good news is that the main ideas of these notes stay mostly intact. We still write down a class of simplified descriptions of the data, still measure quality by a squared error, and still minimise. The unifying statement, which is the thread running through this whole section, is that both methods we study solve

$$\min_{\hat{X} \in \mathcal{C}} \ \|X - \hat{X}\|_F^2$$

for two different choices of the set $\mathcal{C}$ of allowed reconstructions $\hat{X} \in \mathbb{R}^{n \times d}$. In both cases $\hat{X}$ is a *simplified stand-in* for the data: row i of $\hat{X}$ is what we keep of the point x_i, and the objective asks that we lose as little as possible.

- *Principal component analysis* takes $\mathcal{C}$ to be the matrices whose rows all lie in one K-dimensional linear subspace of $\mathbb{R}^d$. Each point is replaced by its projection onto a common subspace: the data is compressed in *direction*.
- *K-means* takes $\mathcal{C}$ to be the matrices whose rows take at most K distinct values. Each point is replaced by the centroid of its group: the data is compressed in *identity*, since after the reconstruction all we remember of x_i is which of K representatives it was closest to.

The parameter K measures, in both cases, how much we allow ourselves to keep, and it is a hyperparameter we must pick: it is not fitted by the optimiser.

> **Remark** (The Frobenius norm)**.** *The norm above is the* Frobenius *norm, which is nothing more exotic than the Euclidean norm of a matrix read as a long vector:*
>
> $$\|A\|_F^2 \ := \ \sum_{i,j} A_{i,j}^2 \ = \ \operatorname{tr}(A^\top A).$$
>
> *When the rows of A are $a_1, \dots, a_n$ this is $\sum_{i=1}^n \|a_i\|^2$, so $\|X - \hat{X}\|_F^2 = \sum_{i=1}^n \|x_i - \hat{x}_i\|^2$ really is the total squared reconstruction error, summed over the samples. We use throughout the* cyclicity of the trace*, $\operatorname{tr}(AB) = \operatorname{tr}(BA)$ whenever both products are defined; see Section 2.2.*

Remark (A third index)**.** *Both methods need to count* groups *(clusters, or directions kept), a third kind of index alongside the samples i and the coordinates j used throughout these notes. We use k for it, ranging over $1, \dots, K$, exactly as k already ranged over the classes in Section 6. So x_i is a sample, $x_{i,j}$ one of its coordinates, and μ_k or v_k the k-th centroid or direction.*

7.2 Principal component analysis

Centring. PCA is about *directions of variation*, so the position of the cloud is irrelevant and must be removed first. Throughout this subsection we assume the data has been *centred*: writing $\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$ for the empirical mean, we replace each x_i by $x_i - \bar{x}$, so that

$$\frac{1}{n}\sum_{i=1}^{n} x_i = 0,$$

and we keep calling the resulting matrix X. This is an important step: without it the first principal direction is dominated by the direction of $\bar{x}$, which says where the data sits rather than how it varies. Whether one should further *standardise* the coordinates is a separate and more debatable question, discussed at the end of the subsection.

Projecting onto a subspace. We look for a linear subspace $S \subseteq \mathbb{R}^d$ of dimension $K \leq d$ on which to project the data. Describe it by an orthonormal basis $v_1, \dots, v_K$, that is, $\langle v_k, v_{k'} \rangle = \delta_{k,k'}$, and gather the basis into a matrix

$$V = (v_1 \mid \cdots \mid v_K) \in \mathbb{R}^{d \times K}, \qquad \text{so that} \qquad V^\top V = I_K,$$

the last identity being exactly the statement that the columns are orthonormal. Two matrices are then worth reading carefully, because everything follows from them.

The matrix $XV \in \mathbb{R}^{n \times K}$ collects the *coordinates of the data in the new basis*:

$$XV = \begin{pmatrix} \langle x_1, v_1 \rangle & \dots & \langle x_1, v_K \rangle \\ \vdots & & \vdots \\ \langle x_n, v_1 \rangle & \dots & \langle x_n, v_K \rangle \end{pmatrix}.$$

Its i-th row is the compressed version of x_i: K numbers instead of d. These are the *scores*, or *principal components*, of the data.

The matrix $XVV^\top \in \mathbb{R}^{n \times d}$ puts them back into the original space:

$$XVV^\top = \begin{pmatrix} - & \sum_{k=1}^{K} \langle x_1, v_k \rangle\, v_k & - \\ & \vdots & \\ - & \sum_{k=1}^{K} \langle x_n, v_k \rangle\, v_k & - \end{pmatrix},$$

whose i-th row is the orthogonal projection of x_i onto S. So $P_V \coloneqq VV^\top \in \mathbb{R}^{d \times d}$ is the orthogonal projector onto S, and $\hat{X} = XVV^\top$ is precisely the reconstruction of Section 7.1, with $\mathcal{C}$ the set of matrices whose rows lie in a common K-dimensional subspace.

Exercise 7.1 ($\star$)**.** Check that $P_V = VV^\top$ satisfies $P_V^\top = P_V$ and $P_V^2 = P_V$, so it is indeed an orthogonal projector, and that $\operatorname{rank}(P_V) = K$. Where is $V^\top V = I_K$ used? Note that $VV^\top \neq I_d$ unless $K = d$: the order of the product matters enormously.

The objective, and its two readings. PCA chooses the subspace minimising the reconstruction error:

$$\min_{V \in \mathbb{R}^{d\times K},\ V^\top V = I_K} \|X - XVV^\top\|_F^2.$$

The following computation, which is the heart of the matter, turns this into an eigenvalue problem.

Proposition 7.2 (Reconstruction error and retained variance)**.** *For every $V \in \mathbb{R}^{d\times K}$ with $V^\top V = I_K$,*

$$\|X - XVV^\top\|_F^2 \;=\; \|X\|_F^2 \;-\; \operatorname{tr}\left(V^\top X^\top X\, V\right).$$

Since $\|X\|_F^2$ does not depend on V, minimising the reconstruction error is the same problem as maximising $\operatorname{tr}(V^\top X^\top X V)$.

Proof. Expand the square using $\|A\|_F^2 = \operatorname{tr}(A^\top A)$:

$$\|X - XVV^\top\|_F^2 = \operatorname{tr}(X^\top X) - 2\operatorname{tr}\left(X^\top X V V^\top\right) + \operatorname{tr}\left(VV^\top X^\top X V V^\top\right),$$

where the cross terms were merged using $\operatorname{tr}(A^\top) = \operatorname{tr}(A)$. For the last term, cyclicity of the trace moves the leading V to the end,

$$\operatorname{tr}\left(VV^\top X^\top X V V^\top\right) = \operatorname{tr}\left(V^\top X^\top X V\, V^\top V\right) = \operatorname{tr}\left(V^\top X^\top X V\right),$$

using $V^\top V = I_K$; and the same cyclicity gives $\operatorname{tr}(X^\top X V V^\top) = \operatorname{tr}(V^\top X^\top X V)$. The two therefore combine into $-2 + 1 = -1$ times $\operatorname{tr}(V^\top X^\top X V)$, which is the claim. □

The quantity being maximised has a direct statistical meaning. Since the data is centred,

$$\operatorname{tr}\left(V^\top X^\top X V\right) = \sum_{k=1}^{K} v_k^\top X^\top X\, v_k = \sum_{k=1}^{K}\sum_{i=1}^{n} \langle x_i, v_k\rangle^2 = \|XV\|_F^2,$$

which is n times the total *empirical variance of the projected data.* So the two natural things one might have asked for turn out to be the same request:

$$\begin{array}{c} \textit{keep the subspace} \\ \textit{losing the least information} \end{array} \quad\Longleftrightarrow\quad \begin{array}{c} \textit{keep the subspace} \\ \textit{carrying the most variance.} \end{array}$$

This is the first thing to remember about PCA. It is also why the empirical covariance matrix

$$\hat{\Sigma} \;\coloneqq\; \frac{1}{n} X^\top X \;\in \mathbb{R}^{d\times d}$$

is the object that decides everything: it is symmetric positive semi-definite (see Section 2.1), and the problem is now to maximise $\operatorname{tr}(V^\top \hat{\Sigma} V)$ over orthonormal V.

Solving it. Start with a single direction, where the answer is the classical Rayleigh quotient bound.

Lemma 7.3 (One direction)**.** *Let $A \in \mathbb{R}^{d\times d}$ be symmetric with eigenvalues $\lambda_1 \geq$*

$\cdots \geq \lambda_d$ *and corresponding orthonormal eigenvectors* $u_1, \ldots, u_d$*. Then*

$$\max_{\|v\|=1} \; v^\top A v = \lambda_1,$$

attained at $v = u_1$*.*

Proof. Write $v = \sum_{j=1}^{d} c_j u_j$ in the eigenbasis, so that $\|v\|^2 = \sum_j c_j^2 = 1$. Then $v^\top A v = \sum_j \lambda_j c_j^2 \leq \lambda_1 \sum_j c_j^2 = \lambda_1$, a weighted average of the eigenvalues being at most the largest. Equality requires all the weight to sit on eigenvalues equal to λ_1, and $v = u_1$ works. □

The general case is the same statement one dimension at a time, and the proof is pleasantly short once one thinks in terms of the projector rather than the basis.

Theorem 7.4 (PCA)**.** *With* $A = X^\top X$ *symmetric positive semi-definite, with eigenvalues* $\lambda_1 \geq \cdots \geq \lambda_d \geq 0$ *and orthonormal eigenvectors* $u_1, \ldots, u_d$*,*

$$\max_{V^\top V = I_K} \; \operatorname{tr}\left(V^\top X^\top X V\right) = \lambda_1 + \cdots + \lambda_K,$$

attained by $V = (u_1 \mid \cdots \mid u_K)$*. The optimal subspace is therefore the span of the* K *leading eigenvectors of* $X^\top X$*, equivalently of the empirical covariance* $\hat{\Sigma}$*, and the minimal reconstruction error is* $\lambda_{K+1} + \cdots + \lambda_d$*. The vectors* $u_1, u_2, \ldots$ *are called the* principal directions*, and* u_k *the* k*-th one.*

Proof. Put $B = VV^\top$, the orthogonal projector onto the chosen subspace, and set $\alpha_j := u_j^\top B\, u_j$ for each j. Two observations:

$$\alpha_j = \|V^\top u_j\|^2 \geq 0, \qquad \alpha_j \leq \|u_j\|^2 = 1,$$

the second because an orthogonal projection never lengthens a vector, and

$$\sum_{j=1}^{d} \alpha_j = \operatorname{tr}(B) = \operatorname{tr}(VV^\top) = \operatorname{tr}(V^\top V) = \operatorname{tr}(I_K) = K.$$

Now expand $A = \sum_j \lambda_j u_j u_j^\top$ and use cyclicity again:

$$\operatorname{tr}\left(V^\top A V\right) = \operatorname{tr}(AB) = \sum_{j=1}^{d} \lambda_j \, u_j^\top B u_j = \sum_{j=1}^{d} \lambda_j \alpha_j.$$

So the problem is bounded by the largest value of $\sum_j \lambda_j \alpha_j$ over all $\alpha \in [0,1]^d$ with $\sum_j \alpha_j = K$. Since the λ_j are sorted, the best possible allocation of a total budget K into weights capped at 1 is to give a full unit to each of the K largest, giving $\lambda_1 + \cdots + \lambda_K$. Taking $V = (u_1 \mid \cdots \mid u_K)$ gives exactly $\alpha_j = \mathbf{1}\{j \leq K\}$, so the bound is attained. Finally the minimal reconstruction error follows from Proposition 7.2 and $\|X\|_F^2 = \operatorname{tr}(X^\top X) = \sum_j \lambda_j$. □

Remark (The solution is nested, and the subspace is what matters)**.** *The proof shows something the statement hides: the optimal* K*-dimensional subspace* contains *the optimal* $(K-1)$*-dimensional one. One does not re-solve the problem for each* K*;*

one computes the eigenvectors once, sorted, and takes as many as one wants. Note also that V itself is not unique (any rotation of an orthonormal basis of the same subspace gives the same reconstruction, and eigenvectors are only defined up to sign), but the subspace *is, as soon as $\lambda_K > \lambda_{K+1}$.*

Remark (In practice one uses the SVD)**.** *Forming $X^\top X$ and diagonalising it is the textbook route but not the numerical one, for the reason given in Appendix A.2: it squares the condition number. Every library instead computes the singular value decomposition $X = UDW^\top$, with $U \in \mathbb{R}^{n\times n}$ and $W \in \mathbb{R}^{d\times d}$ orthogonal and $D \in \mathbb{R}^{n\times d}$ zero off its diagonal, with diagonal entries $s_1 \geq s_2 \geq \cdots \geq 0$. Then $X^\top X = WD^\top DW^\top$, so the principal directions are the columns of W (the right singular vectors) and $\lambda_k = s_k^2$. The scores are $XV = UD$ restricted to the first K columns.*

Choosing K. The eigenvalues answer this too, at least descriptively. The quantity

$$\frac{\lambda_1 + \cdots + \lambda_K}{\lambda_1 + \cdots + \lambda_d} \in [0, 1]$$

is the *fraction of variance explained* by the first K directions, and plotting either it or the individual λ_k against k (a *scree plot*) is the usual way to decide. One looks for an elbow, a K after which the eigenvalues fall off a cliff, or simply takes the smallest K explaining, say, 95% of the variance. Both are rules of thumb rather than theorems: there is no test error here to minimise. When PCA is used as a preprocessing step before a supervised model, however, K becomes an ordinary hyperparameter of that model and can be chosen honestly by cross-validation.

Remark (Centring is not optional, scaling is a modelling choice)**.** *PCA maximises variance, and variance has units: a feature measured in millimetres has a variance 10^6 times larger than the same feature in metres, and will monopolise the first direction for no better reason. When the coordinates are heterogeneous quantities one therefore standardises them first (Appendix A.1), which amounts to diagonalising the empirical* correlation *matrix rather than the covariance. When all coordinates are commensurate, as with the pixels of an image, standardising is usually the wrong thing to do, since the differences in variance are then real information.*

Exercise 7.5 ($\star$)**.** Take $d = 2$ and a cloud of points lying almost exactly on a line through the origin. What are λ_1, λ_2 roughly, what is u_1, and what does the reconstruction $XVV^\top$ look like for $K = 1$? Draw the picture.

Exercise 7.6 ($\star\star$)**.** Show that PCA also solves the following apparently different problem: among all matrices $\hat{X}$ of rank at most K, the one minimising $\|X - \hat{X}\|_F$ is $\hat{X} = XVV^\top$ with V as in Theorem 7.4. (Start by checking that the rows of $XVV^\top$ lie in a K-dimensional subspace, so $\operatorname{rank}(XVV^\top) \leq K$; then argue that any $\hat{X}$ of rank $\leq K$ has its rows in some K-dimensional subspace, and that for a *fixed* such subspace the best $\hat{X}$ is the orthogonal projection of X onto it.) This is a special case of the Eckart-Young theorem.

7.3 K-means

We now compress in the other direction. Instead of asking that all the points be well described by a common subspace, we ask that each point be well described by *one of K representatives*.

The objective. Fix the hyperparameter K, the number of clusters. We look for

- a *partition* of the samples into K groups $S_1, \dots, S_K$, i.e. disjoint sets with $S_1 \cup \cdots \cup S_K = \{1, \dots, n\}$;
- and K *centroids* $\mu_1, \dots, \mu_K \in \mathbb{R}^d$, one per group,

minimising the total squared distance from each point to the centroid of its own group,

$$\mathcal{L}(S, \mu) \;=\; \sum_{k=1}^{K} \sum_{i \in S_k} \|x_i - \mu_k\|^2.$$

This is the natural way to say "the groups should be tight". Note that only the squared Euclidean distance appears, which is already a modelling commitment: it is what will make the clusters come out round.

Matrix form: the same problem as PCA, differently constrained. The partition is more conveniently encoded by an *assignment matrix*. Let

$$Z = (z_{i,k}) \in \{0,1\}^{n \times K}, \qquad \sum_{k=1}^{K} z_{i,k} = 1 \quad \text{for every } i,$$

so that $z_{i,k} = 1$ exactly when sample i belongs to cluster k, each row of Z containing a single 1. Stack the centroids into

$$M = \begin{pmatrix} - & \mu_1 & - \\ & \vdots & \\ - & \mu_K & - \end{pmatrix} \in \mathbb{R}^{K \times d}.$$

Then row i of the product ZM is $\sum_k z_{i,k} \mu_k$, which is simply the centroid assigned to sample i, and therefore

$$\mathcal{L}(M, Z) = \sum_{i=1}^{n} \sum_{k=1}^{K} z_{i,k} \|x_i - \mu_k\|^2 = \|X - ZM\|_F^2.$$

So K-means is

$$\min_{M \in \mathbb{R}^{K \times d}} \; \min_{\substack{Z \in \{0,1\}^{n \times K} \\ \sum_k z_{i,k} = 1 \ \forall i}} \|X - ZM\|_F^2,$$

which is exactly the reconstruction problem of Section 7.1: the reconstruction is $\hat{X} = ZM$, whose rows take at most K distinct values, namely the μ_k. Compare with PCA, where $\hat{X} = XVV^\top$ had rows in a K-dimensional subspace. Same objective, same K, different notion of "simple".

Exercise 7.7 ($\star$)**.** Check the middle equality above in detail: why does $\sum_k z_{i,k}\|x_i - \mu_k\|^2$ equal $\|x_i - \sum_k z_{i,k}\mu_k\|^2$? (Use that exactly one term of the sum is nonzero. Warning: this would be false for a general Z with rows summing to 1 but entries in $[0,1]$ rather than $\{0,1\}$.)

The bad news. Unlike every problem we have met so far, this one is genuinely intractable: minimising $\mathcal{L}$ jointly over Z and M is **NP-hard**, already for $d = 2$. The reason is visible in the formulation: Z ranges over a *discrete* set, of size K^n, and no amount of convexity is going to help, since the objective is not even defined on a continuum. Exhaustive search is hopeless, and there is no closed form and no gradient descent to fall back on.

The good news: alternating minimisation. What saves the situation is that although the joint problem is hard, each of the two variables is easy *given the other*, and in both cases the partial minimisation has a closed form. That suggests the obvious algorithm: minimise alternately in Z and in M. This is *Lloyd's algorithm*, universally called "the" K-means algorithm.

Lemma 7.8 (The two steps)**.** (Assignment.) *For a fixed M, the minimiser over admissible Z is obtained independently for each sample by assigning it to its nearest centroid:*

$$z_{i,k} = 1 \quad \text{if} \quad k = \operatorname*{argmin}_{1 \le k' \le K} \|x_i - \mu_{k'}\|,$$

ties broken arbitrarily.
(Update.) *For a fixed Z, with associated groups $S_k = \{i : z_{i,k} = 1\}$ assumed nonempty, the minimiser over M is obtained independently for each cluster by taking the mean of its points:*

$$\mu_k = \frac{1}{|S_k|} \sum_{i \in S_k} x_i = \frac{\sum_{i=1}^n z_{i,k}\, x_i}{\sum_{i=1}^n z_{i,k}}.$$

Proof. For the assignment step, write $\mathcal{L}(M, Z) = \sum_{i=1}^n \big(\sum_k z_{i,k}\|x_i - \mu_k\|^2\big)$. The constraint couples nothing across samples: each row of Z is chosen separately, and since that row must put its single 1 somewhere, the best place is the smallest of the K numbers $\|x_i - \mu_k\|^2$.

For the update step, the objective splits as $\sum_{k=1}^K \big(\sum_{i \in S_k} \|x_i - \mu_k\|^2\big)$, and each bracket involves only μ_k. So we must minimise $\mu \mapsto \sum_{i \in S} \|x_i - \mu\|^2$ for a fixed set S. This is a quadratic function of μ, with Hessian $2|S|\, I_d \succ 0$, hence strictly convex, so its unique minimiser is the point where its gradient vanishes:

$$\nabla_\mu \sum_{i \in S} \|x_i - \mu\|^2 = -2 \sum_{i \in S} (x_i - \mu) = 2\Big(|S|\,\mu - \sum_{i \in S} x_i\Big) = 0 \quad \Longleftrightarrow \quad \mu = \frac{1}{|S|} \sum_{i \in S} x_i,$$

which is the announced mean. □

Lloyd's algorithm. Initialise the centroids $\mu_1^{(0)}, \dots, \mu_K^{(0)} \in \mathbb{R}^d$. Then for $t = 0, 1, 2, \dots$ repeat:

1. *Assignment.* $Z^{(t+1)} = \operatorname{argmin}_Z \ \mathcal{L}(M^{(t)}, Z)$: assign each sample to its nearest current centroid.
2. *Update.* $M^{(t+1)} = \operatorname{argmin}_M \ \mathcal{L}(M, Z^{(t+1)})$: replace each centroid by the mean of the samples just assigned to it.

Stop when the assignment no longer changes.

Why it stops. The convergence argument is unusual and pleasantly elementary: it is not an analytic statement about small step sizes, but a counting argument.

Proposition 7.9 (Convergence in finitely many steps)**.** *The sequence $\mathcal{L}(M^{(t)}, Z^{(t)})$ is non-increasing and bounded below by 0. Moreover Lloyd's algorithm terminates after finitely many iterations.*

Proof. Each of the two steps minimises $\mathcal{L}$ in one variable with the other held fixed, so neither can increase it:

$$\mathcal{L}(M^{(t)}, Z^{(t)}) \ \geq \ \mathcal{L}(M^{(t)}, Z^{(t+1)}) \ \geq \ \mathcal{L}(M^{(t+1)}, Z^{(t+1)}),$$

and $\mathcal{L} \geq 0$ as a sum of squares. For termination, note that after the update step the centroids are *determined* by the assignment, so from $t \geq 1$ onwards the value $\mathcal{L}(M^{(t)}, Z^{(t)})$ is a function of $Z^{(t)}$ alone. There are at most K^n admissible assignment matrices, hence finitely many possible values. A non-increasing sequence taking finitely many values is eventually constant; and as soon as the value fails to decrease strictly, the assignment step has returned the same Z and the algorithm has stopped by its own criterion. □

Remark (Stopping is not succeeding)**.** *Proposition 7.9 guarantees termination, and nothing more. The point reached is a* local *minimum, in the weak sense that neither step alone can improve it, and it can be arbitrarily worse than the global one: it is easy to draw four obvious clusters and an initialisation from which Lloyd's algorithm splits one of them in two and merges another two, then stops, perfectly stable and perfectly wrong. This is exactly the difficulty that convexity spared us throughout the supervised part of these notes, and there is no way around it here.*
The standard remedies are (i) to run the algorithm several times from different random initialisations and keep the run with the smallest final $\mathcal{L}$, which is cheap and is what libraries do by default, and (ii) to initialise cleverly rather than uniformly at random, spreading the initial centroids out. The latter is the K-means++ initialisation: pick the first centroid uniformly among the data points, then each subsequent one at random with probability proportional to the squared distance to the nearest centroid already chosen. It costs almost nothing and improves results markedly.

What can go wrong. Beyond the local minima, three limitations are worth knowing, all of them consequences of the squared Euclidean distance in the objective.

- *The number of clusters must be given.* K is a hyperparameter, and unlike in the supervised setting there is no held-out error to select it: the objective $\mathcal{L}$ decreases

mechanically as K grows, reaching 0 at $K = n$, so it cannot be used to choose K either. The usual device is again an *elbow plot* of the final $\mathcal{L}$ against K, looking for the point where the curve flattens.

- *The clusters come out round.* Minimising squared distances to a centre means the implied shape is a ball, and the boundary between two clusters is the hyperplane bisecting the segment between their centroids. Two interleaved crescents, or one long thin cluster next to a round one, will be cut in a way no human would choose. If the natural groups are not blobs, K-means is the wrong tool.

- *Outliers hurt twice.* A single far-away point contributes an enormous squared distance, so it drags the centroid of whichever cluster adopts it, and it may even claim a whole cluster for itself. Squared errors are not robust, exactly as in Section 4.

To which one should add the familiar warning that the Euclidean distance mixes the coordinates, so features on wildly different scales must be standardised first.

Exercise 7.10 ($\star$)**.** Why does $\min \mathcal{L}$ decrease as K increases? (Show that any solution with K clusters can be turned into one with $K + 1$ clusters and no larger loss.) Deduce that choosing K by minimising $\mathcal{L}$ would always return $K = n$.

Exercise 7.11 ($\star\star$)**.** Take $n = 4$ points in $\mathbb{R}^2$ at the corners of a rectangle that is much wider than it is tall, and $K = 2$. Find the global minimiser. Then exhibit an initialisation of the centroids from which Lloyd's algorithm converges to the other, worse, partition.

8 Exercises

This section collects additional exercises.

Exercise 8.1 ($\star$)**.** Let $A \in \mathbb{R}^{d\times d}$ and let $f(w) = w^\top A w$. Prove that $f(w) = \frac{1}{2} w^\top (A + A^\top) w$.

Exercise 8.2 ($\star$)**.** All convex functions have a global minimiser. True or False?

Exercise 8.3 ($\star$)**.** Let $f : \mathbb{R}^2 \to \mathbb{R}$ be defined as $f(w_1, w_2) = w_1 w_2$.

1. Draw the level lines of f in the $\mathbb{R}^2$ plane. Using these level lines, prove that f is not convex.
2. Prove that f can be written $f(w) = w^\top A w$ where $A \in \mathbb{R}^{2\times 2}$.
3. Is A unique? If not, provide the corresponding set of matrices A.
4. This time using A, prove that f is not convex.

Exercise 8.4 ($\star$)**.** Let $A \in \mathbb{R}^{d\times d}$ be symmetric with eigenvalues $\lambda_1 \geq \cdots \geq \lambda_d$. Show that for every $w \in \mathbb{R}^d$,

$$\lambda_d \|w\|^2 \;\leq\; w^\top A w \;\leq\; \lambda_1 \|w\|^2,$$

and that both bounds are attained. Deduce that $A \succeq 0$ if and only if all its eigenvalues are non-negative.

Exercise 8.5 ($\star\star$)**.** Let $A \in \mathbb{R}^{d\times d}$ be symmetric with $A \succeq 0$, let $b \in \mathbb{R}^d$, and consider $f(w) = \frac{1}{2} w^\top A w - \langle b, w\rangle$.

1. Show that f is bounded below if and only if $b \in \mathrm{span}(A)$ (the span generated by the columns of the matrix A), and that in that case its set of minimisers is exactly $\{w : Aw = b\}$, an affine space of direction $\mathrm{Ker}(A)$.
2. Exhibit an A and a b in $\mathbb{R}^2$ for which f is convex but has no minimiser.

Exercise 8.6 ($\star\star$)**.** Run gradient descent $w_{t+1} = w_t - \gamma \nabla f(w_t)$ on the quadratic $f(w) = \frac{1}{2}(w - w^\star)^\top A (w - w^\star)$, with $A \succeq 0$ symmetric.

1. Show that $w_t - w^\star = (I - \gamma A)^t (w_0 - w^\star)$.
2. Assume $A \succ 0$ and let $\lambda_1 \geq \cdots \geq \lambda_d > 0$ be its eigenvalues. Show that $w_t \to w^\star$ for every w_0 if and only if $0 < \gamma < 2/\lambda_1$. This is the threshold announced in Section 5.3, which we had admitted.
3. Still for $A \succ 0$, the speed of convergence is governed by $\rho(\gamma) = \max_j |1 - \gamma\lambda_j|$. Show that ρ is minimised at $\gamma^\star = \frac{2}{\lambda_1 + \lambda_d}$, and that then $\rho = \frac{\kappa - 1}{\kappa + 1}$ where $\kappa =$

λ_1/λ_d is the condition number of A. What does a large κ mean geometrically for the level sets of f?

4. Now drop the assumption $A \succ 0$. What happens to the component of $w_0 - w^\star$ lying in $\mathrm{Ker}(A)$?

Exercise 8.7 ($\star$)**.** Fit an intercept explicitly, i.e. minimise $L(w,b) = \frac{1}{2n}\sum_{i=1}^n \left(\langle w, x_i\rangle + b - y_i\right)^2$ over $(w,b) \in \mathbb{R}^d \times \mathbb{R}$.

1. Show that at any minimiser the residuals sum to zero, and that $b^\star = \bar{y} - \langle w^\star, \bar{x}\rangle$, where $\bar{x} = \frac{1}{n}\sum_i x_i$ and $\bar{y} = \frac{1}{n}\sum_i y_i$.

2. Substituting $b^\star$ back, show that $w^\star$ minimises $\frac{1}{2n}\sum_i \left(\langle w, x_i - \bar{x}\rangle - (y_i - \bar{y})\right)^2$. In other words: fitting an intercept is exactly the same as centring the data and forgetting about the intercept.

3. Deduce the closed form in dimension $d = 1$: $w^\star = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$, provided the x_i are not all equal.

Exercise 8.8 ($\star$)**.** Append one extra feature to the data, i.e. replace $X \in \mathbb{R}^{n\times d}$ by $X' = (X \mid v) \in \mathbb{R}^{n\times(d+1)}$ for some $v \in \mathbb{R}^n$. Show that the minimal training loss can only decrease, whatever v is, even if v is pure noise, independent of y. Is the same true of the error on unseen data?

Exercise 8.9 ($\star\star$)**.** Let $0 < \lambda_1 < \lambda_2$ and write w_1, w_2 for the corresponding ridge solutions. By writing the optimality of each one against the other, show that

$$\|w_2\| \le \|w_1\| \qquad \text{and} \qquad L(w_1) \le L(w_2).$$

So increasing λ shrinks the weights and worsens the training loss, monotonically. Why does this not mean that λ should be taken as small as possible?

A Useful Formulas and Good Practices

A.1 Centring and standardising the features

Several places in these notes (gradient descent, ridge regression, PCA, K-means) ask for the features to be rescaled before fitting. Since it is the same operation every time, here it is once and for all.

The operation. For each coordinate $j = 1, \dots, d$, compute the empirical mean and standard deviation of that feature over the training set,

$$\bar{x}_j = \frac{1}{n}\sum_{i=1}^{n} x_{i,j}, \qquad \hat{s}_j = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_{i,j} - \bar{x}_j)^2},$$

and replace each entry of the design matrix by

$$\tilde{x}_{i,j} = \frac{x_{i,j} - \bar{x}_j}{\hat{s}_j}.$$

Every column of the new design matrix then has mean 0 and variance 1. One usually centres the response as well, $\tilde{y}_i = y_i - \bar{y}$. Two caveats on the mechanics: a constant column added for the intercept must be left alone (its standard deviation is 0), and a feature that is constant on the training set should be dropped rather than divided by $\hat{s}_j = 0$. The constants $\bar{x}_j$ and $\hat{s}_j$ must be kept, since a new input has to be put through the very same transformation before the model can be applied to it.

Why: three reasons.

- *Conditioning, hence the speed of gradient descent* (Section 5.3, and the gradient descent exercise of Section 8). The number of GD iterations is governed by the condition number $\lambda_{\max}(X^\top X)/\lambda_{\min}(X^\top X)$, because a single step size γ must be small enough for the steepest direction while progress along the flattest one is set by the smallest eigenvalue. Features measured in wildly different units (a surface in m^2 next to a number of rooms) make that ratio enormous for no reason at all, and gradient descent then crawls.
- *Comparability of the ridge penalty* (Section 5.4). A single λ is applied to every coordinate of w, so it only means the same thing in every direction if the coordinates are on the same scale. Measuring a surface in m^2 rather than km^2 multiplies the corresponding feature by 10^6 and divides its weight by 10^6, so ridge would penalise two encodings of the *same* model completely differently.
- *It costs nothing for ordinary least squares.* The least-squares predictor is unchanged by any invertible linear reparametrisation of the features: replacing X by XA replaces $w^\star$ by $A^{-1}w^\star$ and leaves the predictions $Xw^\star$ alone. So standardising never hurts the fit, which is why it is applied by default.

The trap: compute the statistics on the training data only. This is the mistake that silently inflates every reported number. The constants $\bar{x}_j$ and $\hat{s}_j$ are *estimated from data*, so they are part of the fitting procedure and must be computed on the training portion alone, then applied unchanged to the validation and test portions. Standardising

the full dataset first lets information about the held-out points (their mean, their spread) leak into the fit, and the resulting error estimate is optimistic. The same applies inside cross-validation: the statistics must be recomputed on each of the $K-1$ training folds, not once on the whole set. In practice this is what a "pipeline" object in a machine learning library is for. The rule is worth stating in general: *any* quantity estimated from data is part of the model, and must never see the test set.

A.2 Numerical stability

The formulas of these notes are exact on paper and can overflow on a computer. Two places where this bites, and the standard fixes, all of which are what the libraries do internally.

Log-sum-exp: subtract the maximum. The multiclass loss of Section 6.5 contains $\log \sum_k e^{a_k}$. In floating point e^{a_k} overflows to infinity for a_k around 710 and underflows to zero for a_k around -745, so the expression evaluated naively returns `inf` or $\log 0 = -\infty$ on perfectly reasonable scores. The fix uses the identity, valid for any $c \in \mathbb{R}$,

$$\log \sum_{k=1}^{K} e^{a_k} = c + \log \sum_{k=1}^{K} e^{a_k - c},$$

with the choice $c = \max_k a_k$. Every exponent is then ≤ 0, so no term can overflow, and the largest is exactly $e^0 = 1$, so the sum cannot underflow to zero either. The same trick stabilises the softmax weights $p_k(x)$, which are unchanged by subtracting a constant from all the scores (this is the redundancy of the parametrisation, put to work).

Do not form $X^\top X$. For the normal equations, forming the matrix $X^\top X$ and inverting it squares the condition number of X, so roughly half the available digits of precision are lost before the solve even begins. This is on top of the cost argument of Section 5.3. One instead solves the least-squares problem directly from X, by a QR factorisation or an SVD; this is what a routine such as `numpy.linalg.lstsq` does, and it is the reason to prefer it over an explicit `inv(X.T @ X) @ X.T @ y`.